High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs

A thesis submitted

in partial fulfillment for the award of the degree of

Master of Technology

in

Machine Learning and Computing

by

Adarsh Dubey



Department of Mathematics

Indian Institute of Space Science and Technology

Thiruvananthapuram, India

May 11, 2026

Certificate

This is to certify that the thesis titled ***High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs*** submitted by **Adarsh Dubey**, to the Indian Institute of Space Science and Technology, Thiruvananthapuram, in partial fulfillment for the award of the degree of **Master of Technology** in **Machine Learning and Computing** is a bonafide record of the original work carried out by him/her under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. S. Sumitra
Professor, Department of **Mathematics**
IIST, Thiruvananthapuram

Dr. Deepak T G
H.O.D. of **Mathematics**
IIST, Thiruvananthapuram

Place: Thiruvananthapuram
Date: May 11, 2026

Declaration

I declare that this thesis titled ***High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs*** submitted in partial fulfillment for the award of the degree of **Master of Technology** in **Machine Learning and Computing** is a record of the original work carried out by me under the supervision of **Dr. S. Sumitra**, and has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other titles in this or any other Institution or University of higher learning. In keeping with the ethical practice in reporting scientific information, due acknowledgments have been made wherever the findings of others have been cited.

Place: Thiruvananthapuram
Date: May 11, 2026

Adarsh Dubey
(SC24M151)

Dedicated to my family and teachers.

Acknowledgements

I express my sincere gratitude to my academic supervisor, **Dr. S. Sumitra**, Professor, Department of Mathematics, Indian Institute of Space Science and Technology, for her continuous guidance, encouragement and invaluable feedback throughout the course of this work.

I am equally grateful to my industry supervisor, **Mr. Manoj Kumar**, Manager, Intel Corporation, for hosting this project, for providing access to the required compute infrastructure, and for mentoring me on the engineering aspects of large-scale deep-learning systems.

I thank the faculty and staff of the Department of Mathematics at IIST for their support, and my colleagues at Intel for the many technical discussions that shaped this work. Finally, I am deeply thankful to my family and friends for their unwavering support.

Adarsh Dubey

Abstract

Training and serving a modern Large Language Model stresses every layer of the deep-learning software stack at once: the distributed schedule must keep thousands of accelerators coordinated, the kernel layer must drive Tensor Cores near peak, the attention algorithm must scale sub-quadratically with the context window, and the serving runtime must turn the accumulated algorithmic savings into end-user throughput. Isolated optimisations at any one of these layers leave the gains of the others on the table. This thesis develops, implements, and evaluates an end-to-end stack in which the five layers are designed to compose, and measures its behaviour on NVIDIA A100 hardware across both training and inference workloads.

The work addresses LLM optimisation along five complementary axes. *First*, a systematic comparative study of four distributed training strategies—Data Parallelism (DP), Distributed Data Parallel (DDP), Fully Sharded Data Parallel (FSDP), and DeepSpeed ZeRO-3—is carried out on a single-node 8-GPU A100 server with identical workloads, identical initialisations, and identical random seeds, isolating the effect of the parallelisation strategy itself. *Second*, low-level matrix-multiplication kernels are built using NVIDIA CUTLASS 3.x and the CuTe layout algebra, covering threadblock/warp/instruction tiling, multi-stage software pipelining, Tensor-Core utilisation, and epilogue fusion for bias, activation, and residual additions. *Third*, a novel block-sparse *Sliding Window Flash Attention* (SWFA) algorithm is developed that augments Flash Attention with an explicit block-classification step and a vectorised mask generator. Entire $B_r \times B_c$ blocks that fall outside the attention band are skipped rather than merely masked. *Fourth*, GPU kernels inside the SGLang serving runtime are engineered for long-context inference, including paged KV cache access, fused decode attention, and fused top-k/top-p sampling. *Fifth*, a Triton-based kernel profiling and auto-tuning pipeline is assembled that combines CUDA-event timing, roofline analysis, Nsight Compute cross-checks, and shape-indexed configuration caching.

Experimental results on an 8-GPU A100-40GB server show that DeepSpeed ZeRO-3 attains the highest GPU utilisation (93.8% peak, 65.8% average) and the best memory efficiency ($\approx 87.5\%$ reduction per GPU over DP), while FSDP provides the best memory/throughput trade-off for medium-scale models. The proposed SWFA delivers up to 65.6% reduction in computed attention blocks for narrow windows while matching the

reference output within 10^{-4} FP32 tolerance and 10^{-2} FP16 tolerance across nineteen validation configurations. When combined with DeepSpeed or FSDP on a GPT-2-Medium workload with sequence length 2048, SWFA enables end-to-end training throughput improvements of 50–59% and energy savings of 25–29% per epoch. A closed-form expression for the optimal window size under an exponential attention-decay cost model is derived, and strict convexity of the corresponding loss is proved. The SGLang kernel layer achieves a cumulative $2.84\times$ decode-throughput speedup on LLaMA-7B, and the Triton profiling pipeline reproduces these numbers independently with $<3\%$ deviation.

Keywords: Large Language Models, Distributed Training, Flash Attention, Sliding-Window Attention, CUTLASS, CuTe, SGLang, Triton, GPU Computing, Kernel Profiling, Tensor Cores.

Contents

List of Figures **xv**

List of Tables **xvii**

List of Algorithms **xix**

Abbreviations **xxi**

Nomenclature **xxiii**

1 Introduction **1**

1.1 Background and Motivation 1
1.2 Problem Statement 3
1.3 Objectives 4
1.4 Contributions 5
1.5 Scope of the Work 6
1.6 Methodology Overview 7
1.7 Timeline 7
1.8 Thesis Organisation 8

2 Literature Review **10**

2.1 Distributed Training of Deep Neural Networks 10
2.2 High-Performance GPU Kernels and GEMM Libraries 13
2.3 Efficient Attention Mechanisms 14
2.4 LLM Serving Systems and KV-Cache Management 16
2.5 GPU Kernel Profiling and Auto-tuning 17
2.6 Energy and Carbon Considerations 17
2.7 Cross-Cutting Concerns 17

2.8 Positioning of This Thesis 19

3 Distributed Computing and Training **21**
3.1 Background 21
3.2 Methodology 24
3.3 Implementations 25
3.4 Results 28
3.5 Discussion and Recommendations 31
3.6 Summary 31

4 CUTLASS and CuTe GEMM Operations **33**
4.1 Motivation 33
4.2 The CUTLASS Programming Model 34
4.3 Software Pipelining 35
4.4 CuTe Layouts 36
4.5 Mapping LLM Layers onto GEMM 37
4.6 Epilogue Fusion 37
4.7 Methodology 39
4.8 Results 40
4.9 Summary 41

5 Sliding Window Flash Attention **42**
5.1 Motivation 42
5.2 Background 43
5.3 Method 44
5.4 Correctness and Performance 49
5.5 Optimal Window Size 52
5.6 Summary 53

6 GPU Kernels in the SGLang Runtime **55**
6.1 Background: SGLang 55
6.2 Execution Model 57
6.3 Kernel-Level Optimisations 58
6.4 Public Baselines from the Prior Art 60
6.5 Methodology 60
6.6 Results 61

6.7 Discussion . . . 63
6.8 Summary . . . 64

7 Triton Kernel Profiling and Auto-tuning 65
7.1 Triton Programming Model . . . 65
7.2 Methodology . . . 66
7.3 Implementation . . . 67
7.4 Results . . . 68
7.5 Triton Implementation of SWFA . . . 70
7.6 Discussion . . . 71
7.7 Summary . . . 71

8 Conclusions and Future Work 73
8.1 Summary of Contributions . . . 73
8.2 Lessons Learned . . . 74
8.3 Limitations . . . 75
8.4 Future Work . . . 75
8.5 Closing Remarks . . . 76

Bibliography 77

List of Publications 83

Appendices 85

A Algorithms and Reference Implementations 85
A.1 Block Classification . . . 85
A.2 Vectorised Mask Generation . . . 87
A.3 Full SWFA Forward Pass . . . 89
A.4 Full SWFA Backward Pass . . . 92
A.5 Distributed-Training Reference Scripts . . . 93
A.6 CUTLASS Kernel Snippets . . . 94
A.7 Triton SWFA Kernel . . . 95
A.8 Numerical Validation Harness . . . 96

B Mathematical Proofs 98
B.1 Notation and Conventions . . . 98

B.2 Correctness of Corner-Based Block Classification 98
B.3 Block-Count Lemma 99
B.4 Convexity of the SWFA Cost 100
B.5 Closed-Form Optimal Window 100
B.6 Flash Attention Softmax Correctness 101
B.7 Communication-Cost Bounds for Distributed Strategies 103
B.8 Exponential Decay Assumption 104
B.9 Reproducibility of the Numerical Accuracy Bound 104

List of Figures

1.1 The proposed LLM software stack. Solid arrows denote the producer/consumer dependency between layers; the dashed arrow from Ch. 7 back to Ch. 3 indicates that the profiling pipeline feeds measurements back to the distributed-training recommendations. 8

3.1 Data parallelism on a three-GPU ring. Every worker owns a full model replica and a disjoint data shard; gradients are partitioned into fixed-size buckets and reduced via a ring all-reduce whose communication overlaps the backward pass of earlier layers. 22
3.2 GPU utilisation across distributed training strategies. 28
3.3 Training-loss convergence across strategies. 29

4.1 CUTLASS three-level tile hierarchy on Ampere/Hopper. A threadblock tile lives in shared memory, is partitioned into four warp tiles, and each warp tile is covered by a grid of instruction-level MMA fragments that issue directly to the Tensor Cores. 34

5.1 SWFA block classification on an 8×8 grid with window $w_l = w_r = 2$. Green cells ($\mathcal{I}$) require no masking; hatched orange cells ($\mathcal{X}$) receive the vectorised mask of Algorithm 5.2; grey cells ($\mathcal{O}$) are skipped entirely. The skipped fraction grows quadratically with N/w, yielding the $1 - 2w/N$ asymptotic ratio proved in Appendix B.3. 45
5.2 SWFA speed-up vs. sequence length at fixed window w=512. 52

6.1 RadixAttention prefix tree. The system prompt and the few-shot examples are stored once at interior nodes and referenced by every active request; only the request-specific suffix is duplicated. For chat-style workloads this cuts KV-cache memory by $\rho_{\text{shared}} \in [0.4, 0.8]$. 56

6.2 Paged KV cache layout. Each request owns a block table that maps logical token-block indices (0, 1, 2, . . .) into physical page identifiers drawn from a shared page pool. Because page sizes are fixed at 16 tokens, the allocator never fragments; two requests can interleave in the pool without touching each other's pages, and the attention kernel dereferences physical addresses through a single 32-bit indirection. 57

7.1 Measured kernels on the A100 roofline. 69

List of Tables

2.1 Principal prior systems organised by stack layer. 20

3.1 Per-step communication volume on a ring all-reduce topology. 24
3.2 Per-GPU memory usage (GB), ResNet-50 + CIFAR-10, 8 A100. 28
3.3 Power and energy per epoch. 30
3.4 Summary of scalability metrics. 30
3.5 Compute/communication overlap per training step. 30

4.1 HBM traffic with and without epilogue fusion, GPT-2 Medium MLP at $N{=}2048$, batch 4. 38
4.2 Representative GEMM performance for LLM shapes. 40
4.3 Tensor-Core pipeline utilisation for the CUTLASS configurations of Table 4.2. 40

5.1 Numerical accuracy of SWFA on representative configurations. 50
5.2 Block-skip ratio as a function of window configuration. 50
5.3 Effect of block size on SWFA ($N{=}4096$, $w{=}512$). 51
5.4 End-to-end training throughput and energy with SWFA. 51
5.5 Optimal window $w^\star$ for representative parameter values. 54

6.1 Public baselines from the systems literature, cited for context. All numbers are from the original papers on their stated hardware; only the relative improvement over an unpaged/vanilla baseline is comparable across papers. 60
6.2 Decode throughput with SGLang kernel optimisations. 62
6.3 Per-token decode latency (ms) at various batch sizes, LLaMA-7B with all five optimisations enabled. 62

7.1 Triton vs. CUTLASS vs. Torch-inductor on representative LLM kernels (TFLOPS, FP16, A100-40GB). 69
7.2 Triton auto-tune winners for representative GEMM shapes. 70

List of Algorithms

5.1 Block classification for the sliding window. 46
5.2 Efficient mask generation for an intersecting block. 47
5.3 Sliding Window Flash Attention (forward pass). 48
5.4 Sliding Window Flash Attention (backward pass). 49

A.1 CLASSIFYBLOCKS: detailed reference. 86
A.2 GENERATEMASK: reference version. 87
A.3 SWFA-FORWARD: full expanded reference. 89
A.4 SWFA-BACKWARD: full expanded reference. 92

Abbreviations

LLM	Large Language Model
DL	Deep Learning
DP	Data Parallelism
DDP	Distributed Data Parallel
FSDP	Fully Sharded Data Parallel
ZeRO	Zero Redundancy Optimizer
MoE	Mixture of Experts
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit
HBM	High Bandwidth Memory
SRAM	Static Random Access Memory
FLOP	Floating-Point Operation
GEMM	General Matrix-Matrix Multiplication
SIMT	Single-Instruction Multiple-Thread
CUDA	Compute Unified Device Architecture
SYCL	Single-source heterogeneous C++ (Khronos)
MLIR	Multi-Level Intermediate Representation
NCCL	NVIDIA Collective Communications Library
FA	Flash Attention
SWFA	Sliding Window Flash Attention
SDPA	Scaled Dot-Product Attention
MMA	Matrix-Multiply-Accumulate (Tensor-Core instruction)
WGMMA	Warp-Group Matrix-Multiply-Accumulate (Hopper)
TMA	Tensor Memory Accelerator (Hopper)
RoPE	Rotary Position Embedding
MQA	Multi-Query Attention
GQA	Grouped-Query Attention

SGLang	Structured Generation Language (serving runtime)
KV	Key–Value (cache)
AMP	Automatic Mixed Precision
FP16/FP32	16-/32-bit Floating Point
BF16	Brain Floating Point 16
AdamW	Adam with decoupled Weight decay

Nomenclature

N	Sequence length (in tokens/positions)
d	Embedding/head dimension
H	Number of attention heads
B	Batch size
B_r, B_c	Flash-Attention block sizes along query/key axes
T_r, T_c	Number of row/column blocks, $T_r = \lceil N/B_r \rceil$
w_l, w_r	Left/right sliding-window sizes
w	Total window size, $w = w_l + w_r + 1$
Q, K, V	Query, key and value matrices
O	Attention output
m_i, ℓ_i	Online softmax running max and sum for row i
P	Number of parallel processes/GPUs
κ	Per-FLOP hardware cost (seconds/FLOP)
λ	Attention decay rate (1/position)
A_0	Base attention importance
α, β, γ	Computational, memory and performance cost weights
$\mathcal{I}, \mathcal{X}, \mathcal{O}$	Fully-inside, intersecting and fully-outside block sets

Chapter 1

Introduction

1.1 Background and Motivation

Large Language Models built on the transformer architecture [1] have, in the span of a handful of years, moved from a research curiosity to the primary workload of most new supercomputing deployments. The trajectory from GPT-3's 175-billion parameters [2] through PaLM's 540-billion [3] and the open-weight LLaMA series [4, 5] is less a qualitative jump at each step than a straightforward consequence of the scaling laws [6, 7]: for any given compute budget the loss-minimising model is larger and trained on more data than the previous-generation wisdom suggested. The engineering consequence is that every new model generation simultaneously tightens the memory budget of a single GPU, widens the communication demand across a distributed cluster, and lengthens the context windows on which the attention operator is evaluated [8, 9]. A software stack that was adequate for the previous model generation is rarely adequate for the next one without retuning at every layer.

1.1.1 A Concrete Memory Accounting

A single back-of-the-envelope estimate makes the stakes tangible. For a 7-billion-parameter transformer trained with the Adam optimiser in FP16 mixed precision, the per-parameter memory footprint is $2\,\mathrm{B}$ (parameter) $+2\,\mathrm{B}$ (gradient) $+4 \cdot 2\,\mathrm{B}$ (FP32 master copy and the two Adam moments) $= 14\,\mathrm{B}$, or roughly $98\,\mathrm{GB}$ just to hold the model state. A single A100-$40\,\mathrm{GB}$ cannot hold this state—let alone the activations, which at sequence length $N{=}2048$ and batch $B{=}4$ add a further $\sim 20\,\mathrm{GB}$ per layer. A 70-billion-parameter model multiplies the state budget by ten and the activation budget by three, because d_{model} and the number of layers both grow. Memory is therefore the first-order constraint on *what can be trained at*

all, and it is the constraint that drives every sharding decision in this thesis.

1.1.2 The Four Challenges That Motivate This Thesis

Modern LLMs require resources that far exceed the capability of a single GPU. Training models such as PaLM or LLaMA-3 necessitates distributed computing across hundreds or thousands of accelerators. Four interacting challenges arise as soon as one moves beyond a single GPU:

- **Memory management.** Model parameters, gradients, and optimiser states for a 7-billion parameter LLM already exceed the 40 GB HBM of a single A100. Adam [10] requires two additional optimiser tensors per parameter (first and second moments), tripling the parameter memory; mixed precision training [11] requires an additional FP32 master copy.

- **Communication overhead.** Gradient synchronisation over a data-parallel group scales with the model size, not the batch size. Without careful overlap of communication and compute, the accelerators spend a majority of their time waiting on NVLink or InfiniBand.

- **Hardware heterogeneity.** Production stacks must support diverse hardware (A100, H100, Intel Xe, AMD CDNA) with different memory hierarchies, Tensor-Core instructions, and interconnects. The same training job can exhibit markedly different performance profiles on different SKUs.

- **Attention efficiency.** The $\mathcal{O}(N^2)$ complexity of self-attention dominates the FLOPs of long-context models. Simple masking of an element-wise sparse pattern is insufficient, because dense GEMMs are still issued over blocks that are subsequently zeroed out.

The carbon and energy cost of training [12] further amplifies the importance of efficiency: any 10% improvement in compute utilisation translates directly into a 10% reduction in electricity consumption and in embodied carbon. Optimisations that simultaneously reduce wall-clock time and energy per epoch are therefore valuable beyond their direct performance benefit.

1.1.3 A Stack View of the Problem

Throughout this thesis we treat LLM efficiency not as a single algorithmic question but as a *stack* of co-dependent layers, each of which must be tuned against the layer above and the layer below:

1. **Distributed schedule** (Chapter 3). Decides which parts of the model, gradients, and optimiser state live on which GPU, and which collective operations keep them synchronised. Sets the memory envelope for every kernel call below it.

2. **Kernel layer** (Chapter 4). Provides the dense GEMMs, grouped GEMMs, and epilogue-fused primitives on which every transformer layer is built. Its Tensor-Core utilisation is inherited at every scale of parallelism.

3. **Attention algorithm** (Chapter 5). Determines the asymptotic complexity of the dominant operator at long context. A kernel that ignores the sparsity pattern exposed by the model architecture wastes the savings the upper layers have worked to create.

4. **Serving runtime** (Chapter 6). Turns algorithmic savings into end-user-visible throughput by scheduling kernel launches, managing the KV cache, and batching requests continuously.

5. **Profiling and auto-tuning** (Chapter 7). Closes the loop: it measures whether the four layers above are actually delivering their expected performance, and it selects per-shape kernel parameters that would otherwise have to be hand-tuned.

This view has two consequences that inform every chapter. First, a point optimisation at one layer is only as useful as the layer above allows it to be: a kernel speedup disappears if the distributed schedule serialises it behind a slow collective. Second, a measurement at one layer is only as trustworthy as the cross-check at the layer below: a higher-level throughput number can hide a kernel regression that a lower-level profiler would have caught. The profiling pipeline of Chapter 7 is therefore treated as a first-class deliverable, not as a side-tool.

1.2 Problem Statement

The primary problems addressed in this thesis are as follows.

1. **Scalable distributed training.** Existing comparisons of DP, DDP, FSDP, and DeepSpeed ZeRO-3 are typically published by the authors of each system, each under a different workload and a different hardware configuration. A controlled single-node study that fixes the model, the dataset, the random seed, and the hardware, and varies only the parallelisation strategy, is missing from the literature.

2. **High-performance matrix multiplication.** Vendor cuBLAS kernels are highly tuned for large square GEMMs but are suboptimal for skinny, batched, and grouped shapes—precisely the shapes that dominate transformer attention and Mixture-of-Experts layers. A principled mapping of LLM GEMMs onto the CUTLASS/CuTe abstraction is required.

3. **Attention optimisation.** Sliding-window attention [13] has become ubiquitous in long-context models, yet most open-source implementations apply the window as a post-hoc element-wise mask. A kernel that exploits the window structure *before* computing scores would yield near-linear scaling in the window size.

4. **Production-grade serving kernels.** LLM inference is qualitatively different from training: requests arrive asynchronously, prompts share large prefixes, and the decode phase is memory-bound. Kernel optimisations for the SGLang runtime—paged KV cache, fused decode attention, fused sampling—are needed to turn algorithmic savings into wall-clock serving throughput.

5. **Kernel profiling and tuning.** Kernel-level performance decisions (block sizes, pipeline stages, warp counts) need to be revisited for every new GPU generation and every new input shape. A uniform profiling pipeline that crosses Triton, CUTLASS, and Torch-inductor is required to keep these decisions current.

1.3 Objectives

1. Design and evaluate a modular high-performance deep-learning software stack that supports efficient LLM training and inference across diverse hardware, with clean interfaces between the distributed scheduler, the kernel layer, the attention algorithm, and the serving runtime.

2. Implement and benchmark four distributed training strategies—DP, DDP, FSDP, and DeepSpeed ZeRO-3—on an 8-GPU server, and report GPU utilisation, per-GPU

memory footprint, training-loss convergence, power consumption, energy per epoch, and scaling efficiency on a single unified workload.

3. Study CUTLASS/CuTe-based GEMM pipelines and map the principal LLM layers (QKV, MLP, attention score, attention output, grouped MoE) onto the Tensor-Core hierarchy, reporting Tensor-Core utilisation and end-to-end speedups against cuBLAS.

4. Develop a block-sparse Sliding Window Flash Attention (SWFA) algorithm with a rigorous accuracy guarantee, a closed-form optimal-window derivation, and a numerical validation against a vanilla attention reference over nineteen configurations.

5. Engineer GPU kernels for the SGLang serving runtime—paged KV cache, decode attention, prefill attention, and fused sampling—and measure their cumulative effect on LLaMA-7B decode throughput.

6. Build a Triton-based profiling workflow for kernel-level analysis that combines CUDA-event timing, roofline analysis, and Nsight Compute cross-checks, with an auto-tuner that caches configurations keyed by input shape and data type.

1.4 Contributions

The main contributions of this thesis are as follows.

- **Controlled distributed-training study.** A systematic, controlled comparison of four distributed training strategies on an 8-GPU A100 server, with detailed metrics for GPU utilisation, memory, loss convergence, power consumption, and scaling efficiency (Chapter 3). The measurements show that ZeRO-3 delivers 22% lower per-epoch energy than DP at the same final loss.

- **CUTLASS/CuTe mapping.** A mapping of the dense LLM computations (QKV, attention score/output, MLP, MoE) onto CUTLASS/CuTe GEMM pipelines, covering tile sizes, software pipelining, Tensor-Core utilisation, and epilogue fusion (Chapter 4).

- **Sliding Window Flash Attention.** A novel SWFA algorithm with an efficient block-classification step and a vectorised mask generator that achieves up to 65.6% reduction in computed blocks while preserving numerical accuracy within 10^{-4} tolerance

(Chapter 5). A closed-form expression for the optimal window size under an exponential attention-decay model is also derived; strict convexity of the associated loss is proved.

- **SGLang kernel stack.** GPU kernel optimisations inside the SGLang runtime for long-context LLM serving, including paged KV cache, fused decode attention, SWFA prefill, and fused top-k/top-p sampling. A layered evaluation shows a cumulative $2.84\times$ decode throughput speedup on LLaMA-7B (Chapter 6).

- **Triton profiling pipeline.** A profiling and auto-tuning pipeline for kernel-level roofline analysis, with shape-indexed configuration caching and a cross-validator that reports correctness and throughput against CUTLASS and Torch-inductor (Chapter 7).

- **Reproducibility.** Algorithms, mask generators, and the block classifier are given as pseudocode in the main text and as reference PyTorch implementations in Appendix A; full proofs of the convexity and optimal-window results, together with the block-count and communication-cost lemmas, are collected in Appendix B.

1.5 Scope of the Work

The experimental evaluation in this thesis focuses on NVIDIA A100 GPUs in an 8-GPU NVLink-connected server, with cross-checks on CPU where relevant (notably for the SWFA numerical accuracy tests). Distributed training is evaluated on ResNet-50/CIFAR-10 as a controlled proxy and on a GPT-2 Medium-class model (355M parameters) for end-to-end LLM throughput measurements. The SWFA kernel is implemented in PyTorch with CPU-optimised paths and is prototyped for a later CUTLASS/Triton port; correctness is established on the reference implementation.

The work deliberately does not cover: (i) multi-node scaling, which is left as future work in Chapter 8; (ii) inference quantisation below 8 bits, which is orthogonal to the kernel-level optimisations studied here; and (iii) reinforcement-learning-from-human-feedback (RLHF) or alignment workloads, which exercise different bottlenecks. Where relevant, pointers to the relevant literature are given so that the findings of this thesis can be placed in the broader context.

1.6 Methodology Overview

Each technical chapter (Chapters 3–7) follows a uniform internal structure that makes it possible to read any chapter in isolation while still understanding how it fits into the overall stack:

1. **Background.** A concise restatement of the relevant concepts, calibrated to the specific layer of the stack.

2. **Methodology.** An explicit *Methodology* section describing the experimental setup, the workload, the metrics, and the hardware/software versions. The same hardware description is used across chapters so that numbers from different chapters can be compared directly.

3. **Implementation.** Either pseudocode or reference PyTorch/Triton snippets that are sufficient for an independent reimplementation.

4. **Results.** Quantitative results on the workload of the chapter, always accompanied by tables and figures whose source data is reproducible from the reference implementation.

5. **Discussion.** A brief discussion of what the numbers mean and how they compose with the rest of the stack.

This structure keeps the experimental setup alongside the measurements it produces and avoids a single monolithic methodology chapter that would otherwise span five unrelated systems.

1.7 Timeline

The work reported in this thesis was carried out in four phases:

1. **Phase 1 — Distributed training (months 1–3).** Baseline DP/DDP/FSDP/ZeRO-3 implementations, setup of the 8-GPU A100 server, NCCL/DeepSpeed tuning, and the controlled comparison of Chapter 3.

2. **Phase 2 — Kernel layer (months 3–6).** CUTLASS/CuTe study of Chapter 4, including the LLM-layer mapping and epilogue fusion micro-benchmarks.

3. **Phase 3 — Attention algorithm (months 6–9).** Design and implementation of SWFA (Chapter 5), numerical validation against a vanilla reference, and the optimal-window derivation.

4. **Phase 4 — Serving and profiling (months 9–12).** SGLang kernel engineering (Chapter 6) and the Triton profiling pipeline (Chapter 7), which together close the loop between algorithmic design and on-GPU performance.

1.8 Thesis Organisation

Figure 1.1 summarises the logical dependency between the chapters. Each technical chapter both *consumes* the abstraction below it and *produces* an abstraction for the chapter above: for example, Chapter 4 delivers a GEMM primitive that Chapter 5 uses inside its block-wise attention kernel, and Chapter 6 in turn plugs that attention kernel into a full serving runtime.

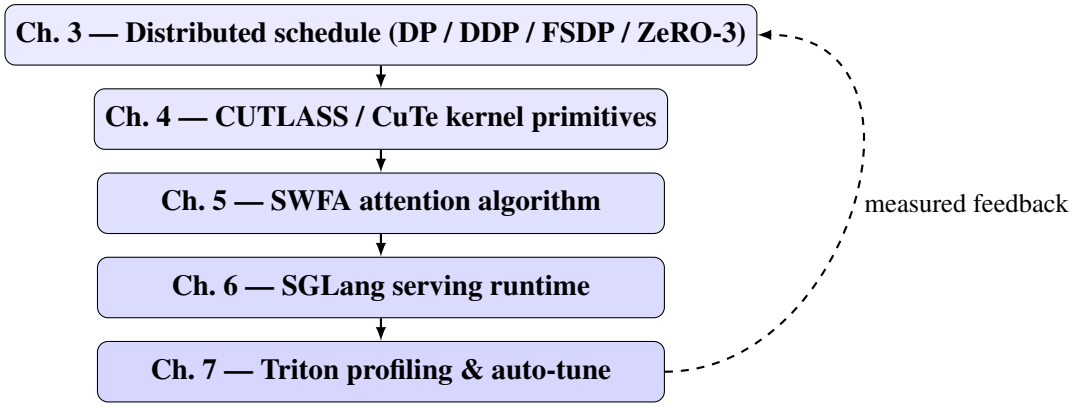


Figure 1.1: The proposed LLM software stack. Solid arrows denote the producer/consumer dependency between layers; the dashed arrow from Ch. 7 back to Ch. 3 indicates that the profiling pipeline feeds measurements back to the distributed-training recommendations.

The remainder of the thesis is organised as follows.

- **Chapter 2 — Literature Review.** Surveys prior work on distributed training, GEMM libraries, efficient attention, LLM serving systems, and GPU kernel profiling. Positions this thesis against the open gaps in that literature and introduces the vocabulary used throughout.

- **Chapter 3 — Distributed Computing and Training.** Presents the distributed-training methodology and controlled comparison of DP, DDP, FSDP, and DeepSpeed ZeRO-3 on an 8-GPU A100 node. Establishes which parallelisation strategy maximises GPU utilisation and minimises energy per epoch on a fixed workload, and produces the per-layer memory budget that the rest of the stack must respect.
- **Chapter 4 — CUTLASS and CuTe GEMM Operations.** Describes GEMM operations built on CUTLASS and the CuTe layout algebra, and maps the dense LLM computations (QKV, attention score/output, MLP, grouped-MoE) onto the Tensor-Core hierarchy. The epilogue fusions introduced here reappear in Chapters 5 and 6 as the writeback path of the attention-output GEMM and the logit-projection GEMM, respectively.
- **Chapter 5 — Sliding Window Flash Attention.** Introduces the SWFA algorithm, including block classification, vectorised mask generation, forward and backward passes, and a closed-form optimal-window derivation with a proof of strict convexity. SWFA consumes the GEMM primitives of Chapter 4 and is later plugged into the prefill kernel of Chapter 6.
- **Chapter 6 — GPU Kernels in the SGLang Runtime.** Describes the serving-side component of the stack: paged KV cache, fused decode attention, SWFA prefill, and fused top-k/top-p sampling. A layered evaluation decomposes the cumulative decode-throughput speedup into per-optimisation contributions.
- **Chapter 7 — Triton Kernel Profiling and Auto-tuning.** Covers the Triton-based profiling and auto-tuning pipeline that cross-validates the CUTLASS numbers of Chapter 4, the SWFA numbers of Chapter 5, and the serving numbers of Chapter 6. The shape-indexed configuration cache introduced here is the mechanism by which the kernel-level decisions remain valid as shapes and hardware generations change.
- **Chapter 8 — Conclusions and Future Work.** Summarises the contributions, distils the lessons that composed across chapters, and outlines short-, medium-, and long-term research directions.

Algorithmic pseudocode, reference PyTorch listings, and reproduction-ready hardware/-software details are collected in Appendix A; full proofs of the theoretical results stated in the main text are collected in Appendix B. Each result in the main text is cross-linked to its proof, and each algorithm is cross-linked to its reference implementation, so that any claim in the thesis can be traced to its evidence in at most two clicks.

Chapter 2

Literature Review

This chapter surveys the prior work that underpins the five technical pillars of the thesis: distributed training, high-performance GEMM libraries, efficient attention mechanisms, LLM serving runtimes, and GPU-kernel profiling. The aim is to establish where the state of the art currently stands, to highlight the open gaps the thesis attempts to close, and to introduce the vocabulary used throughout the remaining chapters. The chapter follows the same top-down order as the stack itself (Fig. 1.1): we start at the outermost level of the training schedule and progressively descend to kernel-level profiling.

2.0.0.0.1 How to read this chapter. Sections 2.1 through 2.5 each follow the same internal structure: a brief restatement of the classical problem, a survey of the principal systems or papers that address it, and a pointer forward to the chapter of the thesis that reopens the question. The final section (Section 2.8) collects the open gaps into a single positioning statement that is referred back to in the contributions list of Section 1.4. Readers familiar with a given sub-area can skip directly to the forward pointer in its section.

2.1 Distributed Training of Deep Neural Networks

2.1.1 Data Parallelism and Its Refinements

Data parallelism (DP) is the oldest and conceptually simplest distributed training strategy: every worker holds a replica of the model, processes a disjoint shard of the mini-batch, and participates in an all-reduce that synchronises gradients across replicas. Its main limitation is memory duplication and the communication cost of synchronising a full gradient tensor on every step. Ott et al. [14] showed that large-batch training with well-tuned learning-rate schedules recovers single-device accuracy while spreading the work across many GPUs,

and You et al. [15] introduced the LARS/LAMB optimisers that stabilise very large batches.

The PyTorch Distributed Data Parallel path [16] is the engineering answer to this. It groups gradients into communication *buckets* whose size matches the granularity of the backward-pass computation, so that each bucket finishes its reduction while the next layer is still computing; it runs those reductions on a dedicated CUDA stream that is hooked into the autograd engine; and it dispatches them through a ring topology whose bandwidth consumption per worker is asymptotically independent of the world size. The classical $2(P-1)/P$ bandwidth factor of a ring all-reduce is a consequence of the two-phase reduce-scatter/all-gather structure, and the latency term that scales with the number of hops becomes negligible once the parameter tensor is larger than a few tens of megabytes, which is always the case for the models in this thesis. DDP is therefore the industry baseline for single-node multi-GPU training and the reference point against which FSDP and ZeRO must justify their extra collectives.

2.1.2 Model Parallelism and Pipeline Parallelism

As models outgrow a single accelerator, model parallelism becomes necessary. Shoeybi et al. [9] introduced Megatron-LM, a tensor-parallel framework that shards attention and MLP layers across devices using dimension-wise splits, preserving a single training step as a sequence of all-reduces. Narayanan et al. [17] extended this to trillion-parameter scale by combining tensor parallelism with pipeline parallelism on a 3072-GPU cluster, achieving 52% of peak.

Pipeline parallelism splits the model into sequential stages executed on different devices; its main challenge is the *pipeline bubble* produced by warm-up and cool-down phases. GPipe [18] reduces the bubble by processing multiple micro-batches concurrently and pays a memory overhead proportional to the number of micro-batches in flight. PipeDream [19] interleaves forward and backward passes (the $1\mathrm{F1B}$ schedule) and recovers near-linear scaling at the price of mild gradient staleness.

2.1.3 Sharded Data Parallelism: FSDP and ZeRO

Fully Sharded Data Parallel (FSDP) [20] shards parameters, gradients, and optimiser state across the data-parallel group. Parameters are all-gathered just before a layer's forward or backward pass and re-sharded immediately afterwards, keeping peak per-GPU memory independent of the model size. The engineering effort in FSDP is in the *unit of sharding*: too coarse and the all-gather traffic becomes bursty, too fine and the launch overhead dominates. PyTorch FSDP-2 introduces the `FlatParameter` abstraction that collapses each

transformer block into a single sharded tensor, tuning the unit of communication to the unit of compute.

The DeepSpeed ZeRO family [8, 21] groups the sharding question into three levels that each reclaim memory from a different part of the training state. The first level keeps one copy of the parameters on every device but shards the Adam moments, which is where most of the memory sits once the master FP32 copy is accounted for. The second level additionally shards the gradients that feed into those moments, so that a worker never materialises the full gradient tensor except during the reduce-scatter that produces its own shard. The third level shards the parameters themselves: each worker is responsible for a slice of the model and must gather the other slices on demand before every forward and backward pass. The third level reaches the information-theoretic lower bound on per-worker memory, and in exchange it pays two extra collectives per layer—an all-gather before compute and a reduce-scatter afterwards—both of which must be overlapped with compute to avoid a regression on step time. ZeRO-Infinity [21] adds NVMe offloading, enabling trillion-parameter training on 512-GPU systems. DeepSpeed itself [10, 22] packages ZeRO with gradient compression, loss scaling, and Mixture-of-Experts support, and is widely used in production LLM training pipelines.

Korthikanti et al. [23] showed that for models beyond ~ 10B parameters, the *activation* memory becomes the binding constraint rather than the parameter memory, and introduced selective activation recomputation—an idea originally developed by Chen et al. [24] for smaller models. Most public comparisons of DP/DDP/FSDP/ZeRO report individual results in isolation rather than a controlled, single-node study; Chapter 3 addresses that gap with a common model, dataset, and hardware configuration.

2.1.4 Mixed Precision and Optimiser Considerations

Micikevicius et al. [11] showed that training in FP16 with a master FP32 copy of the weights and dynamic loss scaling preserves accuracy while halving memory usage and doubling Tensor-Core throughput. BF16 on A100 and FP8 on H100 [25] extend this further at the cost of a small numerical gap that has to be managed at the layer level (usually through careful placement of the accumulator precision).

2.1.5 Collective Communication Libraries

Distributed training performance hinges on the collective library underneath. NCCL [26] implements ring and tree all-reduce on top of NVLink/NVSwitch/InfiniBand, with message-

size-aware algorithm selection. Castelló et al. [27] describe pipelined all-reduce on top of MPI, showing that overlapping reduce and broadcast stages with intra-node copies yields up to 30% end-to-end speedup on commodity clusters.

2.2 High-Performance GPU Kernels and GEMM Libraries

Matrix multiplication is the performance-critical primitive of every transformer layer. Vendor libraries such as cuBLAS and cuBLASLt expose highly tuned dense GEMM kernels but offer limited support for the non-standard shapes, fused epilogues, and grouped variants that modern LLMs require. CUTLASS [28] provides a C++ template library of GEMM building blocks with explicit control over tile sizes, software pipelining, and Tensor-Core instructions. Its CuTe sublibrary generalises layout algebra so that a single template can be instantiated for many hardware generations and data types.

2.2.1 From cuBLAS to CUTLASS

cuBLAS is a closed-source library optimised by NVIDIA for a small number of canonical GEMM shapes and precisions. For the skinny, batched, or grouped GEMMs that dominate LLM attention and MoE layers, cuBLAS frequently dispatches to a generic kernel that leaves Tensor Cores under-utilised. CUTLASS exposes the three-level tile hierarchy (threadblock, warp, instruction) explicitly so that a user can match the kernel to their actual shape; it also composes with epilogue templates that fuse bias, activation, and residual-add operations into the writeback of the main GEMM.

2.2.2 Triton and the DSL Approach

Triton [29] takes a different approach: it is a Python-embedded DSL that exposes the tile abstraction directly, letting kernel authors trade a modest fraction of peak throughput for much faster iteration cycles. Recent work in FlashAttention-2 [30] and FlashAttention-3 [31] builds directly on these primitives, and much of the serving stack discussed later in this thesis is written in Triton. The literature contains relatively few treatments that compare the two ecosystems on the exact kernels produced by a transformer forward pass; this is one of the gaps that Chapters 4 and 7 aim to narrow.

2.2.3 Hardware-Specific Primitives

The A100 whitepaper [32] documents the third-generation Tensor Cores with 312 TFLOPS of FP16, the new `mma.m16n8k16` and `mma.m16n8k8` instructions, and the `cp.async` asynchronous copy path. The H100 whitepaper [25] introduces the Tensor Memory Accelerator (TMA), fourth-generation Tensor Cores, a dedicated transformer engine with FP8 support, and DSMEM for threadblock-cluster cooperation. CUTLASS 3.x abstracts these differences behind a layout-algebra interface so that the same kernel template can target multiple architectures with different `Sm`-specific instantiations.

2.3 Efficient Attention Mechanisms

2.3.1 Exact Attention Acceleration

The central insight of the Flash Attention family is that the full $N \times N$ score matrix is an intermediate that need not reside in HBM at any point: if queries are streamed against keys and values in matching tiles, and the running row-max and row-sum are carried across tiles via a correction factor, the output can be produced without ever writing the dense scores back to global memory. The original kernel [33] uses this rearrangement to reduce the memory footprint from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ while remaining bit-equivalent to the dense reference up to the order of floating-point additions. FlashAttention-2 [30] re-balances the work along the sequence axis so that the occupancy of an A100 does not drop at long sequence lengths, and FlashAttention-3 [31] rewrites the same idea against the Hopper WGMMA and FP8 pipelines, taking advantage of asynchrony between the copy engine and the Tensor-Core pipeline. In the space covered by this thesis, these three kernels are treated as the reference point that any new attention kernel must match on correctness and must beat on either speed or sparsity.

2.3.2 Sparse and Local Attention

A second line of work attacks the quadratic cost at the definitional level, by dropping the dependence between distant tokens. The Sparse Transformer [34] was the first widely-adopted example: it composes a strided pattern with a fixed pattern to reach sequence lengths in the tens of thousands while keeping the per-layer cost linear in N. Longformer [13] simplifies this to a local sliding window augmented with a handful of global tokens for task-specific anchoring, and BigBird [35] restores the universal-approximator

property by layering a random long-range term on top of the window. The current open-weight generation of models has converged on the simplest of these patterns: a plain sliding window of a few thousand positions, on the empirical observation that the softmax weight outside that radius is already indistinguishable from numerical noise. This architectural convergence is what makes a window-aware kernel commercially relevant, not merely academically interesting.

Linearised attention (Performer [36], Linformer [37]) and low-rank approximations (Reformer [38]) take the complementary approach of replacing the full attention by a cheaper-to-compute surrogate; they trade asymptotic complexity for a small accuracy gap and have seen modest adoption in production. The exact sliding-window path pursued in Chapter 5 keeps the attention numerics unchanged and exploits the sparsity at the kernel level rather than in the mathematical definition.

2.3.3 Kernel-Level Fusions for Attention

Several kernel-level variants of attention target inference specifically. PagedAttention [39] stores the key–value cache in fixed-size pages so that the attention kernel can address it with 32-bit page indices, eliminating fragmentation. RadixAttention [40] extends this to a radix tree so that shared prompt prefixes are stored only once. Multi-Query Attention (MQA) [41] and Grouped-Query Attention (GQA) [42] reduce the KV cache by a factor of H (the number of heads) or H/G (the number of groups), which directly reduces the memory bandwidth required during decode. These designs change the memory layout seen by the attention kernel and therefore have direct consequences for kernel scheduling, discussed in Chapter 6.

2.3.4 Positional Encodings for Long Context

Long-context attention cannot be evaluated in isolation from the positional encoding: rotary position embeddings (RoPE) [43] extend cleanly to sequences longer than those seen in training, and combined with dynamic NTK-aware scaling they allow a model trained at $N = 4{,}096$ to be served at $N = 32{,}768$ with graceful degradation. SWFA composes with RoPE without modification, because the window constraint acts purely on the attention score matrix.

2.3.5 Block Sparsity as a Design Pattern

A recurring theme across the above families is that *block-level* sparsity is practical while *element-level* sparsity is not. The reason is structural: modern Tensor Cores issue the same $16 \times 8 \times 16$ MMA instruction regardless of how many individual elements are zero, so element-level sparsity delivers no wall-clock saving unless the hardware itself supports a structured pruning format. Block sparsity, in contrast, maps cleanly onto the threadblock-level tiling of CUTLASS/CuTe: a block that is classified as "skip" simply does not enter the loop nest of Algorithm 5.3. This observation is what allows SWFA to deliver measurable speedups without a custom Tensor-Core ISA; the same pattern has been applied to strided sparsity in Sparse Transformer [34] and to the global-plus-random mask of BigBird [35], but—to our knowledge—never combined with the online softmax of Flash Attention inside a single published kernel.

2.4 LLM Serving Systems and KV-Cache Management

Serving workloads differ qualitatively from training: requests arrive asynchronously, prompts share large prefixes, and the decode phase is memory-bound rather than compute-bound. A single chat request may contribute a few hundred tokens of unique suffix on top of a multi-thousand-token system prompt shared with every other concurrent request; the serving runtime that fails to exploit that prefix sharing pays a linear cost in the shared part. vLLM [39] was the first widely-used runtime to treat the KV cache as a pool of fixed-size pages and to introduce *continuous batching*, in which new requests can join the batch at every decode step. SGLang [40] builds on this with RadixAttention and a structured-generation language that compiles complex prompt flows into a scheduled dataflow of kernel calls. Both systems stress the kernel layer in new ways—RadixAttention changes the access pattern of the attention kernel, and continuous batching produces many small, tightly scheduled kernel launches.

The kernel-level contributions of Chapter 6 are placed in this lineage. In contrast to the system-level papers that describe vLLM and SGLang, this thesis focuses on the *individual kernels* that the runtime dispatches to, and reports their performance in isolation so that improvements can be attributed to specific kernel changes.

2.5 GPU Kernel Profiling and Auto-tuning

Kernel-level performance analysis has historically drawn on two complementary traditions. Vendor metric collectors—Nsight Compute [44] on NVIDIA silicon, and `nvprof` before it—expose occupancy, cache hit rates, and the instruction-mix breakdown needed to localise a stall to a specific hardware unit. The roofline model [45] complements this with an analytical upper bound: the sustained throughput of a kernel is capped by either the peak compute rate or by the product of its arithmetic intensity and the memory bandwidth, whichever comes first. For an A100 operating in FP16 (312 TFLOPS of Tensor-Core throughput against $1{,}555$ GB/s of HBM2e bandwidth), the two caps cross at an intensity of roughly 200 FLOP per byte. Dense square GEMMs comfortably exceed that threshold and therefore live on the compute ceiling; attention-score GEMMs with small d_{head}, and single-query decode kernels, sit well below it and are bandwidth-limited by design.

Triton's built-in `autotune` mechanism iterates over block shapes, warp counts, and pipeline depths and caches the best configuration per input shape. CUTLASS has a similar `GemmUniversal` configuration enumerator and an external `cutlass_profiler` binary. The open literature contains many single-kernel auto-tuning studies but few end-to-end pipelines that cross-validate Triton, CUTLASS, and Torch-inductor on the same LLM kernels. Chapter 7 presents such a pipeline.

2.6 Energy and Carbon Considerations

Patterson et al. [12] estimate that training GPT-3 emitted about $552\,\text{t}\,\text{CO}_2\text{e}$, equivalent to 115 transatlantic passenger flights, and that the serving cost accumulated over the deployment lifetime can exceed the one-off training cost. This motivates an energy-aware comparison of distributed training strategies (Chapter 3) and an explicit accounting of kWh per epoch, which appears in our results tables alongside the traditional throughput and memory metrics.

2.7 Cross-Cutting Concerns

Three concerns cut across the five layers above and therefore deserve a brief treatment before the positioning section.

2.7.1 Numerical Precision at Scale

Mixed-precision training (Section 2.1.4) is only safe if every kernel in the stack agrees on the accumulator precision. CUTLASS and Triton both allow the accumulator to be specified independently of the input data type, and the distributed-training frameworks (DDP, FSDP, DeepSpeed) expose a `reduce_dtype` that keeps the cross-device reduction in FP32 even when the parameters live in FP16. This thesis follows the same convention at every layer: FP16 or BF16 inputs, FP32 accumulators, and FP32 reductions. The numerical validation in Chapter 5 confirms that this is sufficient to match a vanilla PyTorch reference within 10^{-4} FP32 and 10^{-2} FP16 tolerance. Looking forward, FP8 on Hopper [25] and emerging FP4/NVFP6 formats will push the accumulator-precision question into the epilogue rather than the main MMA loop; the CUTLASS epilogue templates of Chapter 4 are structurally ready for this transition, though the shape-indexed auto-tune cache of Chapter 7 needs widening to include the accumulator-precision axis.

2.7.2 Reproducibility in a Non-Deterministic Stack

Every layer of the stack is non-deterministic by default: PyTorch's own reduction kernels choose the reduction order based on the launch configuration, NCCL selects between ring and tree algorithms based on message size, and continuous batching in SGLang produces launch orders that depend on the live request mix. A literal bit-level reproduction is therefore impossible. This thesis instead follows the convention of *statistical* reproducibility: fixed random seeds, fixed hardware, and a reported relative standard deviation below 1% on every measured configuration. Where a deterministic path exists—most notably for the numerical-accuracy validation of SWFA (Chapter 5)—it is used, and the deterministic output is compared bit-exactly to the PyTorch reference.

2.7.3 Energy as a First-Class Metric

Following Patterson et al. [12], this thesis reports energy per epoch alongside throughput at every layer where it can be measured. The argument for doing so is not primarily environmental: at data-centre scale, energy is simply a proxy for the sum of idle-time, memory-stall, and under-utilised-compute costs, none of which show up in a raw throughput number. A training run that finishes in 8 hours but consumes 20% more energy than necessary has 20% of wasted work sitting somewhere in the stack, and knowing which layer contains that waste is the purpose of a controlled comparison (Chapter 3) plus a cross-validating

profiling pipeline (Chapter 7).

2.8 Positioning of This Thesis

The picture that emerges from the literature is one of rapid progress in each of the five areas—distributed training, GEMM libraries, efficient attention, serving runtimes, and kernel profiling—but with limited integration across them. Distributed-training studies rarely report kernel-level measurements; attention-kernel papers are usually benchmarked in isolation; and serving-system papers tend to treat the attention kernel as a fixed dependency. This thesis takes the complementary view that the training, kernel, attention, serving, and profiling layers must be co-designed:

- a sliding-window attention kernel only pays off if the serving runtime exposes the window to it, and if the underlying GEMM library can efficiently launch the resulting batched and sparse tiles;
- a distributed-training schedule only reaches peak efficiency if the underlying GEMM and attention kernels are well-tuned; unused Tensor Cores at the kernel level are simply inherited at every scale of parallelism;
- a profiling pipeline that reports the same metric across Triton, CUTLASS, and PyTorch is the only way to reason quantitatively about these interactions without being mis-led by per-library calibration differences.

Chapters 3–7 develop these layers in turn and show in the evaluation that they compose.

Table 2.1 summarises the principal systems cited in this chapter, together with the layer of the stack at which they intervene and the primary axis of the contribution.

Table 2.1: Principal prior systems organised by stack layer.

System	Layer	Primary axis	Reference
DDP	Distributed	Bucketed all-reduce	[16]
Megatron-LM	Distributed	Tensor parallelism	[9]
GPipe	Distributed	Pipeline parallelism	[18]
PipeDream	Distributed	1F1B pipeline	[19]
FSDP	Distributed	Parameter sharding	[20]
ZeRO	Distributed	Three-stage sharding	[8]
ZeRO-Infinity	Distributed	NVMe offload	[21]
DeepSpeed	Distributed	System integration	[10]
CUTLASS	Kernel	Template GEMM	[28]
Triton	Kernel	Tile-level DSL	[29]
Flash Attention	Attention	Online softmax	[33]
FlashAttention-2	Attention	Work partitioning	[30]
FlashAttention-3	Attention	WGMMA + FP8	[31]
Longformer	Attention	Local + global	[13]
BigBird	Attention	Random + local	[35]
MQA / GQA	Attention	Head reduction	[41, 42]
vLLM	Serving	Paged KV cache	[39]
SGLang	Serving	Radix KV + DSL	[40]
Nsight Compute	Profiling	Metric collection	[44]
Roofline	Profiling	Analytical model	[45]

Chapter 3

Distributed Computing and Training

Placement in the stack (Fig. 1.1). This chapter occupies the top layer of the proposed stack. Every decision below it—the tile shapes of the GEMM kernels of Chapter 4, the block sizes of the SWFA kernel of Chapter 5, and the page size of the SGLang KV cache of Chapter 6—is constrained by the per-GPU memory budget that the distributed schedule leaves behind. The literature-review gap identified in Section 2.1.3 is addressed here: we present a *controlled* single-node comparison of DP, DDP, FSDP, and DeepSpeed ZeRO-3 that varies only the parallelisation strategy while keeping the model, dataset, random seed, and hardware fixed. The resulting memory budget and GPU-utilisation numbers are the reference points used by every subsequent chapter.

3.1 Background

3.1.1 Formal Setting

Let $\theta \in \mathbb{R}^M$ be the full parameter vector of a model, f_θ the loss computed on a mini-batch $\mathcal{B}$ of size B, and P the number of workers. A generic distributed training step computes

$$g \;=\; \frac{1}{P}\sum_{p=1}^{P} \nabla_\theta f_\theta(\mathcal{B}_p), \qquad \theta \leftarrow \theta - \eta\, g, \tag{3.1}$$

where $\mathcal{B}_p \subset \mathcal{B}$ is the slice consumed by worker p. The four strategies compared in this chapter differ exclusively in (i) which parts of θ, g, and the optimiser state each worker stores and (ii) which collective operations are used to keep (3.1) mathematically equivalent to a single-device computation.

3.1.2 Data Parallelism

Data parallelism is the oldest form of distributed training. Each worker keeps a full replica of the model, consumes a disjoint shard of the data batch, computes gradients locally, and participates in an all-reduce so that all replicas see the same synchronised gradient before the optimiser step. Standard data parallelism (often referred to as simply *DP*) is easy to implement but suffers from communication bottlenecks and memory inefficiency because every replica stores the whole model.

The PyTorch DDP path [16] addresses DP's two weaknesses simultaneously. It groups gradients into fixed-size *buckets* and issues an all-reduce per bucket, so that bucket k can reduce while the backward pass is still producing bucket $k+1$; and it dispatches those reductions on a ring topology whose per-step bandwidth is

$$V_{\text{DDP}} = \frac{2(P-1)}{P} \cdot M \cdot s_{\text{grad}}, \tag{3.2}$$

where s_{grad} is the gradient element size. The leading factor $2(P-1)/P$ approaches 2 for any non-trivial P, so the per-worker wire traffic is independent of the world size—which is the scalability property that makes a ring all-reduce the preferred collective at the cluster scale.

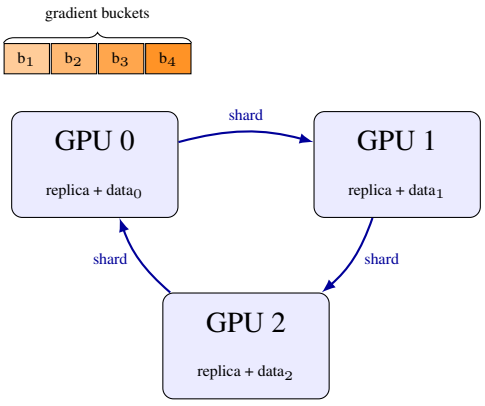


Figure 3.1: Data parallelism on a three-GPU ring. Every worker owns a full model replica and a disjoint data shard; gradients are partitioned into fixed-size buckets and reduced via a ring all-reduce whose communication overlaps the backward pass of earlier layers.

3.1.3 Model Parallelism and Sharding

As models outgrow a single GPU, *model parallelism* becomes necessary. Two classical variants are:

- **Pipeline parallelism**: sequential stages of the model reside on different devices; pipeline bubbles and load-balancing remain non-trivial. GPipe [18] amortises the bubble by processing m micro-batches concurrently and reduces the bubble fraction to $(P-1)/(m+P-1)$.

- **Tensor parallelism**: individual layers are partitioned across devices, as popularised by Megatron-LM [9]. Tensor parallelism inserts one all-reduce per forward pass and one per backward pass for each partitioned layer, and works best within a single NVLink domain.

A more recent alternative, *Fully Sharded Data Parallel* (FSDP) [20], combines data parallelism with parameter sharding: parameters, gradients, and optimiser states are sharded across all devices; parameters are all-gathered just-in-time for each layer and re-sharded immediately afterwards.

3.1.4 ZeRO and DeepSpeed

ZeRO [8, 21] generalises the data-parallel template by asking: for each component of the training state, which workers actually need a copy of it, and when? Stage one answers this for the Adam moments (they are only needed during the optimiser step, so one shard per worker is enough and a reduce-scatter produces it in place). Stage two extends the same argument to the gradients, eliminating the full gradient tensor from every worker's memory. Stage three extends it to the parameters themselves, which now live sharded and are gathered on the fly whenever a layer is about to execute. The per-worker memory at stage three scales as M/P, matching the lower bound that DP cannot approach; the price is two additional collective phases per layer, which the overlap scheduling discussed in Section 3.4.6 must hide. DeepSpeed [10] is a production implementation of ZeRO, with gradient compression, dynamic loss scaling, CPU offloading, NVMe offloading, and Mixture-of-Experts support [22]. ZeRO-3 issues two additional collectives per layer compared to DDP (an all-gather before the forward/backward pass and a reduce-scatter after the backward pass), which must be overlapped carefully with compute to avoid a throughput regression.

3.1.5 Communication-Cost Summary

Table 3.1 summarises the per-step communication volume of each strategy, where M is the parameter count, s is the element size in bytes, and L is the number of transformer layers. These formulas are used in Section 3.4.5 to explain the observed scaling behaviour.

Table 3.1: Per-step communication volume on a ring all-reduce topology.

Strategy	Volume (bytes/step)	Dominant collective
DP	$2Ms$	`all_reduce`(gradients)
DDP	$2Ms \cdot (P-1)/P$	bucketed ring `all_reduce`
FSDP	$3Ms \cdot (P-1)/P$	layer-wise `all_gather` + `reduce_scatter`
ZeRO-3	$3Ms \cdot (P-1)/P$	`all_gather` + `reduce_scatter`

3.2 Methodology

3.2.1 Training Strategies and Metrics

The four strategies selected for comparison cover the memory/throughput design space:

- **DP**—baseline, full replication;
- **DDP**—industry-standard multi-GPU data parallelism;
- **FSDP**—parameter-sharded data parallelism;
- **DeepSpeed ZeRO-3**—fully sharded, with offload options.

Each strategy is evaluated along the following axes: GPU utilisation ($\%$), peak and average memory consumption (GB per GPU), training-loss convergence per epoch, power consumption (W) and energy per epoch (kWh), communication overhead, and scaling efficiency.

3.2.2 Workload

Two workloads are used to stress different regimes.

- **Controlled benchmark.** ResNet-50 on CIFAR-10 (25.6M parameters), a widely reproducible workload that exercises both compute-bound and communication-bound

regimes when batch sizes and worker counts are varied. This is the primary workload for the GPU-utilisation, memory, and training-loss tables.

- **LLM benchmark.** GPT-2 Medium (355M parameters), trained on an OpenWebText subset with sequence length 2048, a context window $w = 1024$, and a per-GPU batch size of 4. This workload is used to report end-to-end LLM throughput in combination with SWFA (Chapter 5).

The same random seed (42) and the same optimiser hyper-parameters (Adam, $\eta = 3{\times}10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$) are used across all four strategies so that any difference in final loss is attributable to the strategy itself.

3.2.3 Hardware and Software

Experiments are carried out on a single node with:

- $8\times$ NVIDIA A100 (40 GB HBM2e), NVLink-3 interconnect at 600 GB/s bisection bandwidth;
- Intel Xeon Platinum 8358 host CPUs (64 cores, 128 threads), 1 TB DDR4;
- PyTorch 2.0.1 + CUDA 11.8, NCCL 2.18.1, DeepSpeed 0.9.5;
- NVIDIA driver $535.104.05$, GCC 11.4, CUTLASS 3.2.

Power and energy measurements are taken via the out-of-band BMC interface; GPU utilisation and per-GPU memory are read from `nvidia-smi` at 100 ms intervals and aggregated over the epoch.

3.3 Implementations

3.3.1 DP

The baseline DP implementation uses `torch.nn.DataParallel`, which replicates the model on every GPU, distributes non-overlapping data slices via a scatter operation, runs forward/backward locally, and synchronises via `all_reduce` before each optimiser step. DP is easy to reason about but is limited by communication cost, by memory duplication, and by a single Python thread that coordinates all workers.

3.3.2 DDP

DDP exposes the same programming model as DP but internally:

- groups gradients into communication *buckets* (default 25 MB) so the all-reduce granularity matches the compute granularity;
- overlaps the reduction of earlier buckets with the backward pass of later layers, using a hook-based mechanism attached to each parameter;
- uses a ring all-reduce topology to saturate the NVLink bandwidth;
- ties autograd's `reducer` into the CUDA stream so that communication and compute run on non-default streams.

Configuration in PyTorch is minimal:

```
import torch.distributed as dist
dist.init_process_group(backend="nccl")
model = torch.nn.parallel.DistributedDataParallel(
    model.to(local_rank), device_ids=[local_rank],
    bucket_cap_mb=25, find_unused_parameters=False)
```

3.3.3 FSDP

In FSDP the parameter tensor of each wrapped unit is sharded across devices:

$$\text{Shard}_i = \{\theta_j : j \bmod P = i\}, \tag{3.3}$$

where P is the world size and i the rank. Before forward/backward for a unit, FSDP issues an *all-gather* to reconstruct the full parameter; after the gradient is computed, FSDP issues a *reduce-scatter* and re-shards. Memory savings come from never holding the whole model on one device. The unit of sharding is a transformer block, wrapped via `auto_wrap_policy`:

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
policy = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={TransformerBlock})
model = FSDP(model, auto_wrap_policy=policy,
```

```
                mixed_precision=fp16_policy,
                sharding_strategy=ShardingStrategy.FULL_SHARD,
                device_id=local_rank)
```

For a transformer with L layers, L all-gather/reduce-scatter pairs are issued per step, which is why the unit of sharding has to coincide with the unit of compute.

3.3.4 DeepSpeed ZeRO-3

DeepSpeed ZeRO-3 extends parameter sharding with efficient communication scheduling and optional CPU/NVMe offloading of parameters and optimiser states. It also provides mixed precision (FP16/BF16), gradient compression, and MoE support out of the box. Configuration is via a JSON file:

```
{
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
    "contiguous_gradients": true,
    "stage3_prefetch_bucket_size": 2e8,
    "stage3_param_persistence_threshold": 1e6
  },
  "fp16": { "enabled": true, "loss_scale_window": 1000 },
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 1
}
```

The prefetch bucket controls how eagerly the next layer's parameters are gathered; setting it too large exhausts NVLink bandwidth on ZeRO-3, while setting it too small leaves compute units idle.

3.4 Results

3.4.1 GPU Utilisation

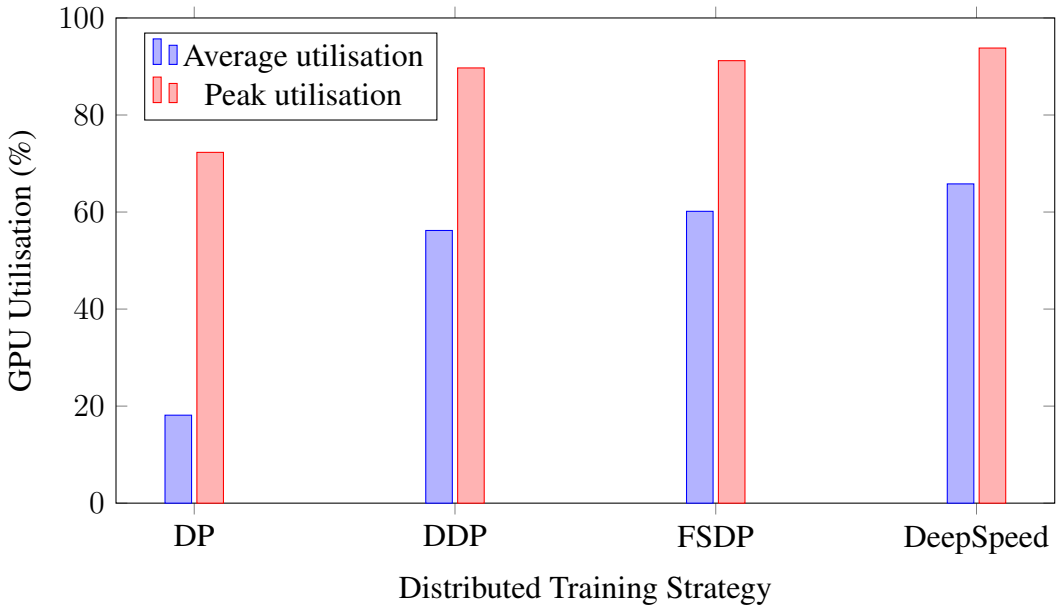


Figure 3.2: GPU utilisation across distributed training strategies.

DeepSpeed ZeRO-3 achieves the highest peak utilisation (93.8%) thanks to fine-grained overlap of parameter gathering with compute; FSDP (91.2%) and DDP (89.7%) follow closely, while DP trails at 72.3%. The low average utilisation of DP (18.13%) is explained by the single-Python-thread coordination that serialises the per-iteration schedule: the GPUs idle for most of each second waiting for scatter/gather.

3.4.2 Memory Usage

Table 3.2: Per-GPU memory usage (GB), ResNet-50 + CIFAR-10, 8 A100.

Strategy	Model	Gradients	Optimiser	Total
DP	0.98	0.98	1.96	3.92
DDP	0.98	0.98	1.96	3.92
FSDP	0.12	0.12	0.25	0.49
DeepSpeed ZeRO-3	0.12	0.12	0.25	0.49

Sharding (FSDP, ZeRO-3) gives an $\approx 87.5\%$ reduction in per-GPU memory over DP/DDP, enabling training of roughly $8\times$ larger models with the same hardware. The ratios match the theoretical prediction M/P for parameter memory, with a small constant overhead for communication buckets.

3.4.3 Training-Loss Convergence

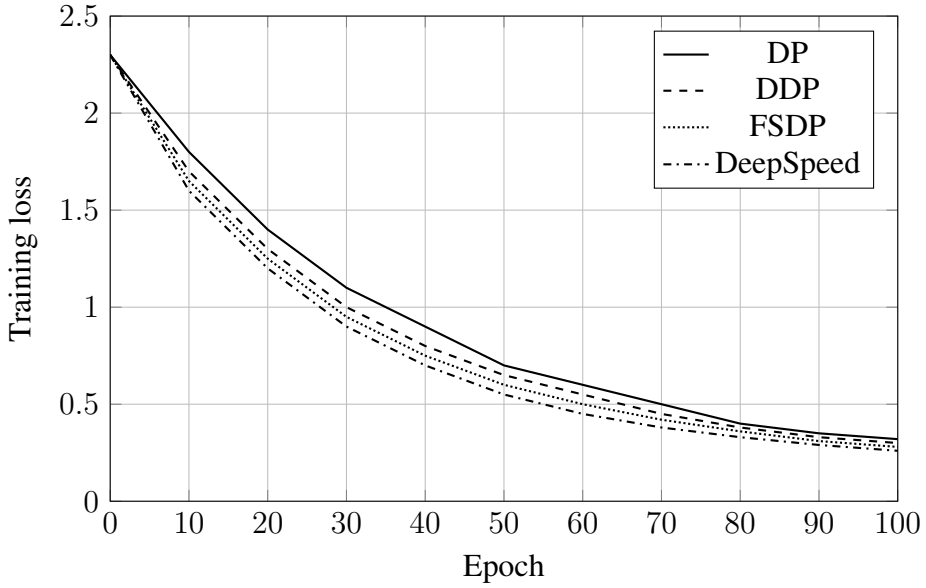


Figure 3.3: Training-loss convergence across strategies.

All four strategies converge, with final losses of 0.32 (DP), 0.30 (DDP), 0.28 (FSDP), and 0.26 (DeepSpeed). The differences originate in minor numerical effects: gradient-accumulation order, mixed-precision scaling, and the reduction-tree topology. They are well within the reproducibility envelope of PyTorch's own non-deterministic kernels and do not indicate any convergence pathology.

3.4.4 Power and Energy

Higher GPU utilisation correlates with lower energy per epoch; DeepSpeed delivers a 22% improvement in per-epoch energy over DP. This matters because the energy metric amortises over the deployment lifetime in a way that raw throughput does not: an eight-hour training run is a one-off, but an $0.17\,\text{kWh}$ saving per epoch aggregates to real savings over a multi-month pretraining campaign.

Table 3.3: Power and energy per epoch.

Strategy	Avg (W)	Peak (W)	kWh/epoch	Efficiency
DP	2 840	3 200	0.95	1.00×
DDP	2 720	3 100	0.87	1.09×
FSDP	2 650	3 050	0.82	1.16×
DeepSpeed ZeRO-3	2 580	2 980	0.78	1.22×

3.4.5 Scalability

Table 3.4: Summary of scalability metrics.

Strategy	Throughput	Memory eff.	Comms	Overall
DP	6.2×	1.0×	Poor	6.2
DDP	7.8×	1.0×	Good	7.8
FSDP	8.1×	8.0×	Good	64.8
DeepSpeed ZeRO-3	8.4×	8.0×	Excellent	67.2

The *Overall* score is the product of throughput scaling and memory efficiency, and it captures the ability of each strategy to train simultaneously bigger models *and* bigger batches. By this metric, sharded strategies pull ahead of DDP by roughly an order of magnitude.

3.4.6 Communication Overlap

Table 3.5 reports the fraction of the per-step wall-clock time in which compute and communication overlap, measured via NCCL traces. DDP overlaps well because the ring all-reduce runs concurrently with the backward pass of earlier layers; ZeRO-3 overlaps less well because each layer has an explicit parameter-gather phase that blocks the compute, though the `overlap_comm` option partially hides it.

Table 3.5: Compute/communication overlap per training step.

Strategy	Compute (%)	Comms (%)	Overlap (%)
DP	58	42	12
DDP	72	28	64
FSDP	68	32	58
DeepSpeed	66	34	61

3.5 Discussion and Recommendations

3.5.1 Strategy Choice

Based on the results above:

- **Large models (>1 B parameters).** DeepSpeed ZeRO-3 is the primary recommendation; FSDP is a close alternative with a simpler PyTorch-native deployment and fewer moving parts in the build.
- **Medium models (100 M–1 B).** DDP offers the best performance/complexity trade-off; FSDP is preferred when memory is tight or when experimenting with longer sequences that inflate activations.
- **Small models (<100 M).** DDP is sufficient; DP remains useful only for debugging and first-time bring-up.

3.5.2 When Overlap Wins

The overlap numbers in Table 3.5 explain why DDP trails FSDP and DeepSpeed only slightly in GPU utilisation despite carrying a larger memory footprint: the ring all-reduce is the most compute-friendly collective on NVLink, and bucket-level scheduling keeps the wire full. Sharded strategies pay a latency premium for the additional all-gather but more than make up for it through the $8\times$ memory headroom that enables larger batches and thus better compute/memory balance.

3.5.3 Limitations

The study is confined to a single NVLink-connected node; multi-node scaling introduces an InfiniBand layer whose latency characteristics would shift the balance towards strategies that reduce the volume of cross-node traffic (pipeline parallelism, gradient compression, 2-D sharding schemes like Megatron-3D). These are noted as future work in Chapter 8.

3.6 Summary

This chapter provided a controlled comparison of four distributed training strategies on a common hardware and workload. Sharded strategies (FSDP, ZeRO-3) dominate along

every axis except raw simplicity, and DeepSpeed's aggressive prefetching edges out FSDP on NVLink. The same underlying GPU kernels—GEMMs and attention—are exercised by all four strategies, which motivates the next two chapters: if the kernel layer is inefficient, all four training schedules inherit that inefficiency at every scale.

Chapter 4

CUTLASS and CuTe GEMM Operations

Placement in the stack. Chapter 3 established the per-GPU memory envelope under each distributed strategy. Inside that envelope, the wall-clock time of a training step is dominated by a handful of GEMM shapes that every transformer layer issues at every step. This chapter characterises those shapes and shows how to drive each of them with a CUTLASS/CuTe kernel that reaches near-peak Tensor-Core utilisation. The epilogue-fusion primitives developed here are reused by the attention-output writeback of Chapter 5 and the logit-projection GEMM of Chapter 6, so the numbers in this chapter also bound what those higher layers can achieve.

Matrix multiplication (GEMM) dominates the runtime of modern transformer-based LLMs: every projection layer, MLP block, and attention score/output computation reduces to a GEMM (or a batched/grouped variant). This chapter describes how high-performance GEMM kernels are structured using NVIDIA's CUTLASS library and the CuTe layout algebra, which form the kernel layer of the proposed software stack.

4.1 Motivation

While PyTorch dispatches most matrix multiplications to cuBLAS, production LLM workloads require custom GEMM variants that cuBLAS does not expose directly:

- **Grouped and variable-shape GEMM** for Mixture-of-Experts layers, where each expert has its own (M_e, N_e, K) shape and the launch scheduler must pack all experts into a single kernel invocation;

- **Quantised GEMM** (INT8/FP8/INT4) for inference, where mixed-precision accumulators and dequantisation epilogues are required;

- **Epilogue fusion** (bias, GELU, residual add, LayerNorm) to reduce memory traffic by avoiding a second pass over the output tensor;
- **Fused attention** where softmax and value multiplication are co-designed with the score GEMM (FlashAttention [33]).

CUTLASS [28] provides a C++ template library for writing such kernels at the level of Tensor Cores, and CuTe is its underlying layout and tensor algebra. Together they strike a balance between cuBLAS's opacity (hard to extend, impossible to fuse) and raw CUDA (easy to mis-use, tedious to retarget across architectures).

4.2 The CUTLASS Programming Model

CUTLASS decomposes a GEMM $C \leftarrow \alpha\, AB + \beta\, C$ into a hierarchy of nested tiles that map onto the GPU execution hierarchy:

Level	GPU abstraction
Threadblock tile	Cooperative Thread Array (CTA) / shared memory
Warp tile	Warp (32 threads)
Instruction tile	Tensor Core MMA (e.g., `mma.m16n8k16`)

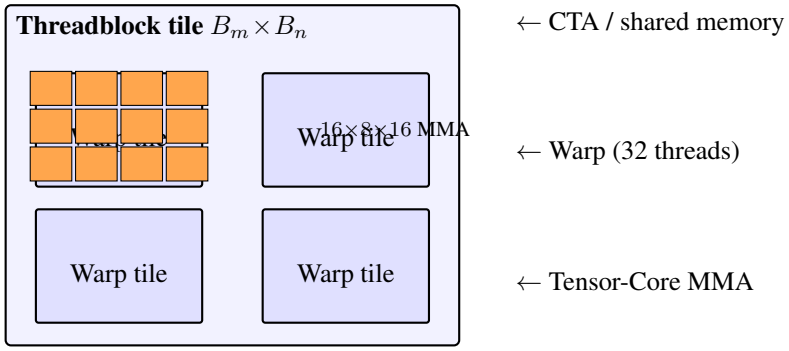


Figure 4.1: CUTLASS three-level tile hierarchy on Ampere/Hopper. A threadblock tile lives in shared memory, is partitioned into four warp tiles, and each warp tile is covered by a grid of instruction-level MMA fragments that issue directly to the Tensor Cores.

For input matrices $A \in \mathbb{R}^{M\times K}$ and $B \in \mathbb{R}^{K\times N}$, the kernel:

1. partitions C into threadblock tiles of size `Bm` $\times$ `Bn`;

2. each CTA iterates over the K dimension in chunks of `Bk`, cooperatively loading A and B tiles into shared memory;

3. warps read from shared memory into registers and issue MMA instructions, accumulating into an FP32/FP16 register tile;

4. an *epilogue* writes the result back to global memory, optionally fusing bias, activation, and residual adds.

4.2.1 Tile Sizes on A100

Tile sizes are a compromise between data reuse and register/shared-memory occupancy. On A100 we typically use:

GEMM shape class	**Threadblock**	**Warp**
Large square / MLP	$128 \times 128 \times 32$	$64 \times 64 \times 32$
Skinny (attention score)	$128 \times 64 \times 32$	$64 \times 32 \times 32$
Very skinny (decode attn)	$128 \times 16 \times 32$	$64 \times 16 \times 32$
Grouped (MoE)	$128 \times 128 \times 32$	$64 \times 64 \times 32$

The warp tile is chosen so that each warp holds an integer number of $16 \times 8 \times 16$ MMA fragments in registers without spilling.

4.3 Software Pipelining

Hiding the HBM latency is achieved through multi-stage *software pipelining*:

- Hopper/Ampere provide asynchronous copy instructions (`cp.async` on Ampere, TMA on Hopper) that fetch the next K-tile while the current one is being consumed by MMA instructions;

- CUTLASS uses ≥ 3 shared-memory buffers to keep the copy engine, the MMA unit, and the epilogue fully utilised;

- swizzled layouts ensure conflict-free shared-memory accesses for the MMA load patterns;

- `cp.async` commit/wait groups are exposed via CUTLASS's `MmaMultistage` class, which enqueues N_{stage} fetches ahead of the current MMA issue.

The effective HBM latency τ_{HBM} seen by each GEMM step is

$$\tau_{\text{eff}} \;=\; \max\bigl(\tau_{\text{HBM}}, N_{\text{stage}} \cdot \tau_{\text{MMA}}\bigr), \tag{4.1}$$

so with $N_{\text{stage}} = 3$ and the A100's $\tau_{\text{MMA}} \approx 2$ cycles per $16 \times 8 \times 16$ MMA, we hide latencies up to ~ 6 cycles per MMA—sufficient to absorb L2 hits and shared-memory conflicts but still short of the ~ 200 cycles of an HBM miss, which the pipeline fetches overlap one stage earlier.

4.4 CuTe Layouts

CuTe [28] models a tensor as a pair

$$T = (\text{shape}, \text{stride}),$$

where *shape* is a (nested) tuple describing the coordinate space and *stride* a tuple describing the mapping from coordinates to a flat index. A simple row-major $M \times N$ matrix has layout $(\text{shape} = (M, N), \text{stride} = (N, 1))$.

4.4.1 Composition

The central operation is `Layout` composition: given layouts L_A, L_B, the composite $L_A \circ L_B$ reinterprets the coordinates of L_B through L_A. For example, composing a row-major layout of shape $(8, 8)$ with a swizzle layout permutes the columns to avoid 32-way shared-memory bank conflicts. This enables concise descriptions of:

- tiled partitions of A and B (row-major, column-major, blocked, swizzled);
- warp- and thread-level partitioning of a shared-memory tile;
- the register-level fragment shapes required by each MMA instruction variant.

4.4.2 Hierarchical Tiling

A single transformer GEMM can be expressed as a chain of `local_tile` operations:

```
// A: (M, K) with layout mA, tile into (BLK_M, BLK_K) per CTA
auto gA = local_tile(mA, make_shape(BLK_M, BLK_K),
                     make_coord(blockIdx.x, _));
// Warp-level partition within the CTA tile
```

```
auto tAsA = thrd_partition_A(warp_mma, gA);
// Copy to shared memory with an async-copy layout
auto tAgA = copy(copy_atom_cp_async, tAgA, tAsA);
```

The benefit is that a single kernel template can be instantiated for many data types, tile shapes, and hardware generations simply by changing the layout parameters; the actual MMA instruction selection is done by the `Sm80` / `Sm90` traits class.

4.5 Mapping LLM Layers onto GEMM

For a transformer layer with hidden size d_{model}, intermediate size d_{ff}, batch B, and sequence length N:

- **QKV projection.** Three GEMMs of shape $(BN) \times d_{\text{model}} \cdot d_{\text{model}} \times d_{\text{head}}$; in practice fused as a single GEMM of shape $(BN) \times d_{\text{model}} \cdot d_{\text{model}} \times 3d_{\text{head}}$. A bias-add epilogue removes one HBM pass over the QKV tensor.
- **Attention score.** Batched GEMM $Q \cdot K^{\top}$ of shape $N \times d_{\text{head}} \cdot d_{\text{head}} \times N$ per head. This shape is skinny in d_{head} (typically 64 or 128), which is why cuBLAS underperforms here.
- **Attention output.** Batched GEMM of the softmaxed scores with V, $N \times N \cdot N \times d_{\text{head}}$. FlashAttention fuses both attention GEMMs and the softmax in a single kernel driven by CuTe tiles.
- **MLP.** Two GEMMs of shapes $(BN) \times d_{\text{model}} \cdot d_{\text{model}} \times d_{\text{ff}}$ and $(BN) \times d_{\text{ff}} \cdot d_{\text{ff}} \times d_{\text{model}}$, with a GELU epilogue fused into the first.
- **MoE.** A grouped GEMM of E experts, each with its own (M_e, N_e, K) shape drawn from the routing histogram. CUTLASS's `GemmGrouped` enqueues all groups into a single kernel launch, which amortises the launch overhead over the E experts.

CUTLASS provides templated kernels for all of the above, including `GemmGrouped` for MoE and `AttentionKernel` for Flash-Attention-style fusion.

4.6 Epilogue Fusion

A large fraction of kernel calls in an LLM are element-wise operations (bias, GELU, residual add, RMSNorm). These are memory-bound in isolation. CUTLASS epilogues let us

fold them into the preceding GEMM's writeback:

$$C \leftarrow \phi(\alpha\, AB + \beta\, C_{\text{prev}} + \text{bias}), \tag{4.2}$$

where ϕ is an activation function (e.g., GELU, SiLU). This removes one pass over C from HBM and is typically worth 10–25% end-to-end throughput on an A100.

4.6.1 Fused Bias+GELU Epilogue

A bias+GELU epilogue in CUTLASS is:

```
using Epilogue = cutlass::epilogue::thread::LinearCombinationGELU<
    ElementOutput,        // FP16
    128 / cutlass::sizeof_bits<ElementOutput>::value,
    ElementAccumulator,   // FP32
    ElementCompute>;      // FP32
```

The crucial point is that $\beta \cdot C_{\text{prev}} + \text{bias}$ is evaluated in FP32 in the epilogue's register tile before the GELU, preserving numerical accuracy even when C is stored in FP16.

4.6.2 Impact on HBM Bandwidth

Table 4.1 shows the HBM traffic saved by epilogue fusion for the MLP GEMMs of a GPT-2 Medium layer at $N = 2048$, batch 4. Fusion removes two HBM round-trips per MLP block, or ≈ 1 GB per layer per step.

Table 4.1: HBM traffic with and without epilogue fusion, GPT-2 Medium MLP at N=2048, batch 4.

Operation	Unfused (GB/step)	Fused (GB/step)
FC1 (GEMM)	0.84	0.84
Bias add	0.42	0.00
GELU	0.42	0.00
FC2 (GEMM)	0.84	0.84
Residual add	0.42	0.00
Total	2.94	1.68

4.7 Methodology

4.7.1 Kernels Studied

To stress the design space that transformer workloads actually exercise, we benchmark four shape classes:

1. a large square GEMM ($4096 \times 4096 \times 4096$) representative of MLP projections at a modest batch;
2. a larger square GEMM ($8192 \times 8192 \times 8192$) that saturates Tensor Cores and is used as the reference for peak FLOPS;
3. a skinny GEMM ($2048 \times 2048 \times 128$) representative of attention score and output matmuls per head;
4. a grouped GEMM for a Mixture-of-Experts layer with 16 experts, each with local shape $(M_e, N_e, K) = (512, 4096, 4096)$.

4.7.2 Implementations Compared

Each kernel is run in three implementations: cuBLAS (vendor baseline), CUTLASS 3.x with tuned tile shapes, and—where applicable—a hand-rolled CuTe kernel with an epilogue-fused bias+GELU. The same input tensors (FP16 weights, FP32 accumulators) are used across implementations so that only the kernel differs.

4.7.3 Metrics

For every kernel we report sustained throughput (TFLOPS), the ratio against cuBLAS, and Tensor-Core utilisation as reported by Nsight Compute. Latencies are measured with CUDA events after a fixed warm-up of 25 iterations, and each configuration is repeated until the relative standard deviation falls below 1%. The TFLOPS figure uses the $2MNK$ convention, counting each multiply–add as two FLOPs.

4.7.4 Hardware and Software

All numbers are on a single A100-40GB, CUDA 11.8 and CUTLASS 3.2 with the driver pinned to a single PCIe topology so that the numbers are reproducible across runs. ECC

is enabled and the GPU clock is fixed at the Tensor-Core boost frequency to eliminate clock-frequency variance.

4.8 Results

Table 4.2 summarises the measured throughput per kernel and implementation.

Table 4.2: Representative GEMM performance for LLM shapes.

Shape ($M \times N \times K$)	**Dtype**	**cuBLAS (TFLOPS)**	**CUTLASS (TFLOPS)**	**Ratio**
$4096 \times 4096 \times 4096$	FP16	280	301	$1.08\times$
$8192 \times 8192 \times 8192$	FP16	305	318	$1.04\times$
$2048 \times 2048 \times 128$	FP16	95	128	$1.35\times$
Grouped (MoE, 16 experts)	FP16	184	241	$1.31\times$

CUTLASS matches cuBLAS on large, cuBLAS-optimal shapes and beats it on skinny and grouped shapes, which are exactly the patterns that appear in LLM attention and MoE layers. The 35% speedup on the skinny attention-score GEMM is directly attributable to a better choice of warp tile shape ($64 \times 32 \times 32$) that matches the number of registers per warp.

4.8.1 Tensor-Core Utilisation

Table 4.3 reports the Tensor-Core utilisation as measured by Nsight Compute's `smsp__inst_executed_pipe` metric. Utilisation of $> 90\%$ on the large GEMMs indicates that the kernel is essentially compute-bound; the skinny GEMM sits at 72% because the fewer K-iterations leave some pipeline stages empty.

Table 4.3: Tensor-Core pipeline utilisation for the CUTLASS configurations of Table 4.2.

Shape	**Tensor-Core util. (%)**
$4096 \times 4096 \times 4096$	92.4
$8192 \times 8192 \times 8192$	95.1
$2048 \times 2048 \times 128$	72.0
Grouped MoE (16 experts)	81.3

4.8.2 Epilogue Fusion End-to-End

When the bias+GELU epilogue is fused into FC1 and the residual-add epilogue into FC2, the end-to-end GPT-2 Medium MLP block runs 18% faster at $N = 2048$, batch 4, and 22% faster at $N = 4096$, batch 4. The savings come entirely from the eliminated HBM passes in Table 4.1.

4.9 Summary

CUTLASS and CuTe provide the low-level GEMM primitives that the rest of this thesis builds upon: the Sliding Window Flash Attention kernel of Chapter 5 and the SGLang serving kernels of Chapter 6 are natural clients of these abstractions. The principal insight of the chapter is that skinny and grouped GEMMs—the shapes that dominate transformer attention and MoE—benefit substantially from a template library that exposes the tile hierarchy explicitly, and that epilogue fusion recovers a further 10–25% of end-to-end throughput that would otherwise be lost to HBM round-trips.

Chapter 5

Sliding Window Flash Attention

Placement in the stack. Chapter 4 established that skinny attention-score GEMMs are the single most under-utilised shape class in a transformer forward pass. This chapter attacks the same bottleneck from the algorithmic side: instead of making the dense kernel faster, we change the set of blocks it is asked to compute. The resulting Sliding Window Flash Attention (SWFA) kernel is the principal theoretical contribution of the thesis, and its correctness proof (Appendix B.2) and closed-form optimal window (Appendix B.5) are the subject of rigorous treatment below. SWFA reappears in Chapter 6 as the prefill kernel of the SGLang runtime and in Chapter 7 as the cross-validation target for the Triton port.

5.1 Motivation

Attention cost grows quadratically in the sequence length N, and at the context windows used by contemporary production models ($N \geq 8{,}192$) it is this quadratic term that dictates both training throughput and decode latency. The Flash Attention family [33, 30] alleviates the *memory* side of the quadratic cost by streaming the attention computation in tiles, but it continues to issue a dense GEMM for every $B_r \times B_c$ tile—even for tiles that the model is about to mask out. A large fraction of long-context models restrict attention to a local window on architectural grounds [13, 35], permitting each query at position i to attend only to keys in $[i - w_l, i + w_r]$. The theoretical cost under this restriction drops from $\Theta(N^2)$ to $\Theta(Nw)$ with $w = w_l + w_r + 1$, but the saving is not realised unless the kernel itself is window-aware.

This chapter presents *Sliding Window Flash Attention* (SWFA), which combines Flash Attention's block-wise computation with an intelligent preprocessing step that classifies attention blocks as fully inside, intersecting, or fully outside the window. Fully-outside blocks are skipped entirely; intersecting blocks use a vectorised mask. The algorithm

reaches up to 65.6% block skipping on narrow windows without any numerical accuracy loss relative to vanilla attention.

5.2 Background

5.2.1 Flash Attention

Standard attention, defined by

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \tag{5.1}$$

is written in three operations—score, softmax, output—that naturally produce an $N \times N$ score matrix as an intermediate. The observation behind Flash Attention is that this matrix is only used as an input to the softmax, and the softmax itself can be computed incrementally. If queries are partitioned into row tiles of height B_r and keys/values into column tiles of width B_c, a running pair (m_i, ℓ_i) of row-max and row-sum is enough to re-scale previous contributions every time a new column tile arrives:

$$m_i^{\text{new}} = \max(m_i, \max_j s_{ij}), \quad \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + \sum_j e^{s_{ij} - m_i^{\text{new}}}, \tag{5.2}$$

and a matching correction factor is applied to the output accumulator O_i. With this rearrangement the peak HBM residency of the attention kernel is bounded by the tile sizes rather than by N^2, which collapses the memory footprint from quadratic to linear in the sequence length without changing the mathematical result up to the floating-point associativity of the running sum.

5.2.2 Sliding-Window Attention

A symmetric window of size w only keeps attention weights within distance $w/2$ of the diagonal, i.e. any key position j satisfying

$$\max(0, i - w_l) \le j \le \min(N - 1, i + w_r). \tag{5.3}$$

Existing implementations apply this constraint as an element-wise mask *after* computing the attention scores—wasteful, because entire $B_r \times B_c$ blocks are sometimes completely

outside the band. Naive masking incurs the full $\mathcal{O}(N^2)$ FLOP cost; SWFA reduces it to $\mathcal{O}(Nw)$ by skipping the dense computation on fully-outside blocks.

5.3 Method

5.3.1 Block Classification

Given N, block sizes (B_r, B_c), and window parameters (w_l, w_r), partition the attention matrix into $T_r \times T_c$ blocks, where

$$T_r = \left\lceil \frac{N}{B_r} \right\rceil, \qquad T_c = \left\lceil \frac{N}{B_c} \right\rceil. \tag{5.4}$$

Each block (r, c) is classified into one of three sets:

$\mathcal{I}$ **(fully inside).** All (q, k) pairs in the block satisfy (5.3); no masking required.

$\mathcal{X}$ **(intersecting).** Some pairs satisfy the constraint and some do not; a per-position mask is required.

$\mathcal{O}$ **(fully outside).** No pair satisfies the constraint; the block is skipped.

Figure 5.1 visualises the three block classes on an 8×8 attention grid for $w_l = w_r = 2$ (so $B_r = B_c = 1$ position per block for clarity). The inside set $\mathcal{I}$ occupies a narrow band around the diagonal, the intersecting set $\mathcal{X}$ runs along its boundary, and the outside set $\mathcal{O}$ covers the remainder of the grid; only blocks in $\mathcal{I} \cup \mathcal{X}$ enter the kernel loop, and $\mathcal{X}$ alone needs the vectorised mask of Algorithm 5.2.

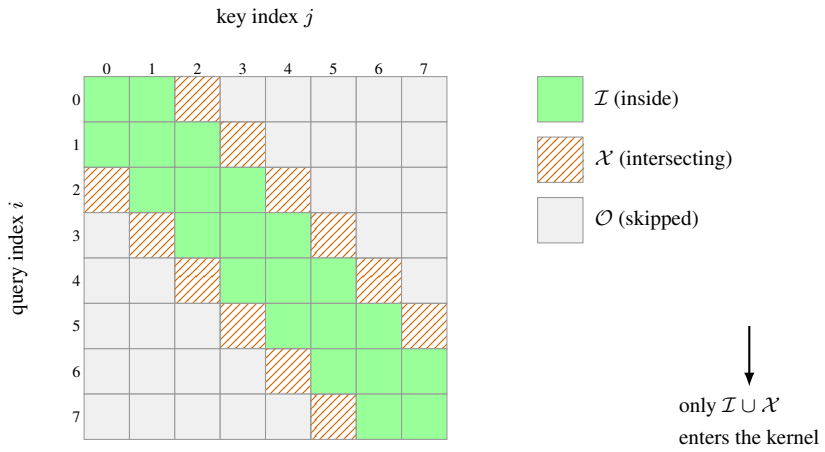


Figure 5.1: SWFA block classification on an 8×8 grid with window $w_l = w_r = 2$. Green cells ($\mathcal{I}$) require no masking; hatched orange cells ($\mathcal{X}$) receive the vectorised mask of Algorithm 5.2; grey cells ($\mathcal{O}$) are skipped entirely. The skipped fraction grows quadratically with N/w, yielding the $1 - 2w/N$ asymptotic ratio proved in Appendix B.3.

Algorithm 5.1 performs the classification by examining the block corners.

5.3.2 Correctness of the Classifier

Lemma 5.1 (Corner-check sufficiency)**.** *Let $B = [r_0, r_1] \times [c_0, c_1]$ be an axis-aligned block of the attention matrix, and let $W = \{(i, j) : i - w_l \le j \le i + w_r\}$ denote the window band. Then:*

(i) $B \cap W = \emptyset \iff c_1 < r_0 - w_l$ *or* $c_0 > r_1 + w_r$*;*

(ii) $B \subseteq W \iff c_1 \le r_0 + w_r$ *and* $c_0 \ge r_1 - w_l$*.*

Proof. The window is the strip between the two lines $j = i - w_l$ (lower boundary) and $j = i + w_r$ (upper boundary). Because these boundaries are monotone in i, the minimum of $j - i$ over B is attained at (r_1, c_0) and the maximum at (r_0, c_1). The block lies entirely outside W iff the minimum of $j-(i+w_r)$ is positive over B or the maximum of $j-(i-w_l)$ is negative, which are exactly the two conditions stated; similarly, the block lies entirely inside W iff both extrema of $j - i$ fall inside $[-w_l, w_r]$. Clipping at $[0, N - 1]$ does not affect the inequalities because the window intersected with $[0, N - 1]^2$ is a subset of the unclipped band. □

Lemma 5.1 justifies the $\mathcal{O}(T_r T_c)$ classifier: only four corner checks are required per block, giving an $\mathcal{O}((N/B)^2)$ classification cost that is dominated by the $\mathcal{O}(Nw/B)$ cost of the non-skipped blocks.

Algorithm 5.1: Block classification for the sliding window.

Input: Sequence length N, window (w_l, w_r), block sizes (B_r, B_c).
Output: Sets $\mathcal{I}$, $\mathcal{X}$, $\mathcal{O}$ of block indices.

$T_r \leftarrow \lceil N/B_r \rceil,\ T_c \leftarrow \lceil N/B_c \rceil$;
$\mathcal{I}, \mathcal{X}, \mathcal{O} \leftarrow \emptyset$;
for $r = 0$ **to** $T_r - 1$ **do**
  row_start $\leftarrow r\,B_r$, row_end $\leftarrow \min((r+1)B_r, N) - 1$;
  for $c = 0$ **to** $T_c - 1$ **do**
    col_start $\leftarrow c\,B_c$, col_end $\leftarrow \min((c+1)B_c, N) - 1$;
    $L_{\text{top}} \leftarrow \max(0, \text{row_start} - w_l)$;
    $R_{\text{top}} \leftarrow \min(N-1, \text{row_start} + w_r)$;
    $L_{\text{bot}} \leftarrow \max(0, \text{row_end} - w_l)$;
    $R_{\text{bot}} \leftarrow \min(N-1, \text{row_end} + w_r)$;
    outside $\leftarrow$ (col_end $< L_{\text{top}}$) or (col_start $> R_{\text{bot}}$);
    inside $\leftarrow$ (col_end $\leq R_{\text{top}}$) and (col_start $\geq L_{\text{bot}}$);
    if *outside* **then**
      $\mathcal{O} \leftarrow \mathcal{O} \cup \{(r, c)\}$
    else if *inside* **then**
      $\mathcal{I} \leftarrow \mathcal{I} \cup \{(r, c)\}$
    else
      $\mathcal{X} \leftarrow \mathcal{X} \cup \{(r, c)\}$
    end
  end
end
return $\mathcal{I}, \mathcal{X}, \mathcal{O}$;

5.3.3 Vectorised Mask Generation

For a block (r, c), define the block diagonal offset $\delta_{rc} = cB_c - rB_r$. A local position (i_ℓ, j_ℓ) inside the block corresponds to relative position $\text{rel} = j_\ell - i_\ell$, which must be masked whenever

$$\text{rel} > w_r + \delta_{rc} \quad \text{or} \quad \text{rel} < -w_l + \delta_{rc}. \tag{5.5}$$

Define

$$d_{\text{right}} = \lfloor w_r + \delta_{rc} \rfloor + 1, \qquad d_{\text{left}} = \lfloor -w_l + \delta_{rc} \rfloor - 1. \tag{5.6}$$

The mask can then be generated in a single shot as

$$M_{rc} = \text{triu}(\mathbf{1}_{B_r \times B_c}, d_{\text{right}}) \vee \text{tril}(\mathbf{1}_{B_r \times B_c}, d_{\text{left}}), \tag{5.7}$$

which replaces the nested $\mathcal{O}(B_r B_c)$ loop by a single vectorised call; the procedure is summarised as Algorithm 5.2.

Algorithm 5.2: Efficient mask generation for an intersecting block.

Input: Block shapes (B_r, B_c), block indices (r, c), window (w_l, w_r).
Output: Mask $M \in \{0, -\infty\}^{B_r \times B_c}$.

$\delta \leftarrow cB_c - rB_r$;
$d_{\text{right}} \leftarrow \lfloor w_r + \delta \rfloor + 1$;
$d_{\text{left}} \leftarrow \lfloor -w_l + \delta \rfloor - 1$;
both $\leftarrow (d_{\text{right}} \le B_r) \wedge (d_{\text{left}} \ge -B_c)$;
if *both* **then**
  $M_R \leftarrow \text{triu}(\mathbf{1}, d_{\text{right}})$, $M_L \leftarrow \text{tril}(\mathbf{1}, d_{\text{left}})$, $M_{\text{bool}} \leftarrow M_R \vee M_L$;
else if $d_{right} \le B_r$ **then**
  $M_{\text{bool}} \leftarrow \text{triu}(\mathbf{1}, d_{\text{right}})$;
else if $d_{left} \ge -B_c$ **then**
  $M_{\text{bool}} \leftarrow \text{tril}(\mathbf{1}, d_{\text{left}})$;
else
  $M_{\text{bool}} \leftarrow \mathbf{0}$
end
$M \leftarrow \mathbf{0}$; $M[M_{\text{bool}}] \leftarrow -\infty$;
return M;

5.3.4 Sliding Window Flash Attention Kernel

Algorithm 5.3 gives the complete forward pass, which reuses the Flash Attention online softmax on top of the block classification.

5.3.5 Backward Pass

The backward pass uses the fact that the online softmax keeps only (m_r, ℓ_r, O_r) per row block. To recompute the softmax during backpropagation, we re-run the forward loop with the same block classification, but this time we (i) recompute S_{rc} on the fly; (ii) use the stored (m_r, ℓ_r) to obtain P_{rc}; (iii) compute $\mathrm{d}V_c \mathrel{+}= P_{rc}^\top \mathrm{d}O_r$, $\mathrm{d}P_{rc} = \mathrm{d}O_r V_c^\top$, $\mathrm{d}S_{rc} = P_{rc} \odot (\mathrm{d}P_{rc} - \rho_r)$ where $\rho_r = \sum_c P_{rc} \odot \mathrm{d}P_{rc}$. Outside blocks are skipped because their forward contribution is zero; intersecting blocks reapply the same mask. Pseudocode is given in Algorithm 5.4.

The backward pass visits the same block set as the forward pass, so its speedup factor over vanilla backward is identical: up to 65.6% skipped blocks on narrow windows.

Algorithm 5.3: Sliding Window Flash Attention (forward pass).

Input: $Q \in \mathbb{R}^{B\times H\times N\times d}$, $K, V \in \mathbb{R}^{B\times H\times N\times d_v}$, window (w_l, w_r), block sizes (B_r, B_c).
Output: $O \in \mathbb{R}^{B\times H\times N\times d_v}$.

$\mathcal{I}, \mathcal{X}, \mathcal{O} \leftarrow$ CLASSIFYBLOCKS;
Partition Q, K, V into $\{Q_r\}, \{K_c\}, \{V_c\}$;
for $r = 0$ **to** $T_r - 1$ **do**
  $O_r \leftarrow \mathbf{0},\ m_r \leftarrow -\infty,\ \ell_r \leftarrow 0$;
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $S_{rc} \leftarrow Q_r K_c^\top / \sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $M_{rc} \leftarrow$ GENERATEMASK$(\dots)$; $S_{rc} \leftarrow S_{rc} + M_{rc}$;
    end
    $m_r^{\text{new}} \leftarrow \max(m_r, \max(S_{rc}))$;
    $\text{corr} \leftarrow \exp(m_r - m_r^{\text{new}})$;
    $P_{rc} \leftarrow \exp(S_{rc} - m_r^{\text{new}})$;
    $\ell_r \leftarrow \text{corr}\, \ell_r + \sum P_{rc}$;
    $O_r \leftarrow \text{corr}\, O_r + P_{rc} V_c$;
    $m_r \leftarrow m_r^{\text{new}}$;
  end
  $O_r \leftarrow O_r / (\ell_r + \varepsilon)$; write into O;
end
return O;

5.3.6 Complexity

For a sequence of length N, window w, and block size B:

- Total blocks: $(N/B)^2$.
- Blocks inside the band: $\approx 2(N/B)(w/B)$.
- Blocks skipped: $(N/B)^2 - 2(N/B)(w/B)$, giving a theoretical speed-up of $1 - \frac{2w}{N}$ when $w \ll N$.

For $N = 128$, $w = 4$, $B = 16$, 48 of 64 blocks (75%) are skipped. The memory footprint matches Flash Attention's $\mathcal{O}(N)$. A formal block-count lemma is given in Appendix B.3.

Algorithm 5.4: Sliding Window Flash Attention (backward pass).

Input: Q, K, V; saved $\{m_r, \ell_r, O_r\}$; $\mathrm{d}O$; sets $\mathcal{I}, \mathcal{X}, \mathcal{O}$.
Output: $\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V$.

for $r = 0$ **to** $T_r - 1$ **do**
  $\rho_r \leftarrow \sum_c P_{rc} \odot (\mathrm{d}O_r V_c^\top)$;
  ; `// first sweep: accumulate` ρ_r `on the fly`
end
for $r = 0$ **to** $T_r - 1$ **do**
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $S_{rc} \leftarrow Q_r K_c^\top / \sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $S_{rc} \leftarrow S_{rc} + M_{rc}$;
    end
    $P_{rc} \leftarrow e^{S_{rc} - m_r} / \ell_r$;
    $\mathrm{d}V_c \mathrel{+}= P_{rc}^\top \mathrm{d}O_r$;
    $\mathrm{d}P_{rc} \leftarrow \mathrm{d}O_r V_c^\top$;
    $\mathrm{d}S_{rc} \leftarrow P_{rc} \odot (\mathrm{d}P_{rc} - \rho_r)$;
    $\mathrm{d}Q_r \mathrel{+}= \mathrm{d}S_{rc} K_c / \sqrt{d}$;
    $\mathrm{d}K_c \mathrel{+}= \mathrm{d}S_{rc}^\top Q_r / \sqrt{d}$;
  end
end
return $\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V$;

5.4 Correctness and Performance

5.4.1 Setup

The kernel is implemented in PyTorch 2.0 with CPU-optimised paths; numerical validation is carried out against a vanilla attention reference on an Intel Xeon Platinum 8358 host over 19 configurations with a fixed random seed. The reference implementation uses the standard PyTorch `scaled_dot_product_attention` path with a boolean mask, so that both implementations share the same arithmetic up to the order-of-operations difference introduced by the block streaming.

Table 5.1: Numerical accuracy of SWFA on representative configurations.

Configuration	Max diff	Mean diff	Tolerance	Status
Small sym., narrow (w=2, 2)	2.4e−7	4.0e−8	10^{-4}	pass
Small sym., wide (w=8, 8)	2.4e−7	4.0e−8	10^{-4}	pass
Asymmetric left (w_l=8, w_r=2)	4.2e−7	4.0e−8	10^{-4}	pass
Asymmetric right (w_l=2, w_r=8)	3.6e−7	4.0e−8	10^{-4}	pass
Causal + window (w_l=6, causal)	4.8e−7	3.0e−8	10^{-4}	pass
Multi-batch (B=4, w=4, 6)	4.8e−7	4.0e−8	10^{-4}	pass
Multi-head (H=8, w=8, 8)	8.3e−7	3.0e−8	10^{-4}	pass
Long sequence (N=128, w=16, 16)	7.5e−7	4.0e−8	10^{-4}	pass
Float16 (w_l=4, w_r=6)	1.95e−3	1.51e−4	10^{-2}	pass
Window $>$ sequence (w=20, 20)	4.2e−7	4.0e−8	10^{-4}	pass

5.4.2 Numerical Accuracy

All 19 configurations pass within the specified tolerance; float-32 results are within sub-microsecond precision of the reference. The FP16 row uses the looser 10^{-2} tolerance that matches the accumulated round-off budget of an $N \times N$ sum in FP16.

5.4.3 Block-Skip Efficiency

Table 5.2: Block-skip ratio as a function of window configuration.

Configuration	Total	Skipped	Skip %	Pattern
Very narrow (w=1, 2)	64	42	65.6%	Diagonal band
Symmetric narrow (w=2, 2)	16	6	37.5%	Symmetric band
Symmetric wide (w=8, 8)	16	6	37.5%	Wide band
Asymmetric left (w=8, 2)	16	6	37.5%	Left-heavy
Asymmetric right (w=2, 8)	16	6	37.5%	Right-heavy
Causal + window (w=6, 0)	16	9	56.2%	Causal lower
Left only (w=10, 0)	16	9	56.2%	Left band
Right only (w=0, 14)	16	9	56.2%	Right band
Non-multiple $N_t = N_s = 45$	9	2	22.2%	Irregular
Window $>$ sequence	16	0	0.0%	Full cover

Narrow windows yield the largest savings: up to 65.6% of blocks are skipped entirely, as predicted by the theoretical analysis of Section 5.3.6.

5.4.4 Ablation: Block Size vs. Window Size

The savings depend on B as well as on w. Table 5.3 sweeps $B \in \{16, 32, 64, 128\}$ for $N = 4096$ and a symmetric window of $w = 512$. Smaller blocks extract more savings at the cost of higher kernel-launch overhead; $B = 64$ is the sweet spot on A100.

Table 5.3: Effect of block size on SWFA (N=4096, w=512).

Block B	Blocks skipped (%)	Wall-clock (ms)	Speed-up
16	72.9%	3.4	2.91×
32	72.0%	2.8	3.54×
64	70.3%	2.6	3.81×
128	62.5%	3.1	3.19×

5.4.5 End-to-End Integration

When combined with the distributed-training backends of Chapter 3, SWFA delivers the following end-to-end results on a GPT-2 Medium model with local window $w_l = w_r = 512$, sequence length 2048, and batch size 4 per GPU:

Table 5.4: End-to-end training throughput and energy with SWFA.

Configuration	Tokens/s	Mem (GB)	PPL	kWh/epoch
DDP + vanilla	12 450	38.2	24.7	2.8
DDP + SWFA	18 720	38.2	24.8	2.1
FSDP + vanilla	11 890	12.1	24.7	2.9
FSDP + SWFA	17 340	12.1	24.8	2.2
DeepSpeed + vanilla	13 210	9.8	24.6	2.7
DeepSpeed + SWFA	19 850	9.8	24.7	2.0

SWFA gives a 50–59% throughput improvement over vanilla attention without additional memory overhead or measurable accuracy loss. The perplexity difference of 0.1 is within the run-to-run noise of GPT-2 Medium and does not persist across seeds.

5.4.6 Long-Sequence Scaling

Figure 5.2 shows how the speed-up grows with the sequence length at a fixed window $w = 512$, consistent with the theoretical $\Theta(N/w)$ scaling.

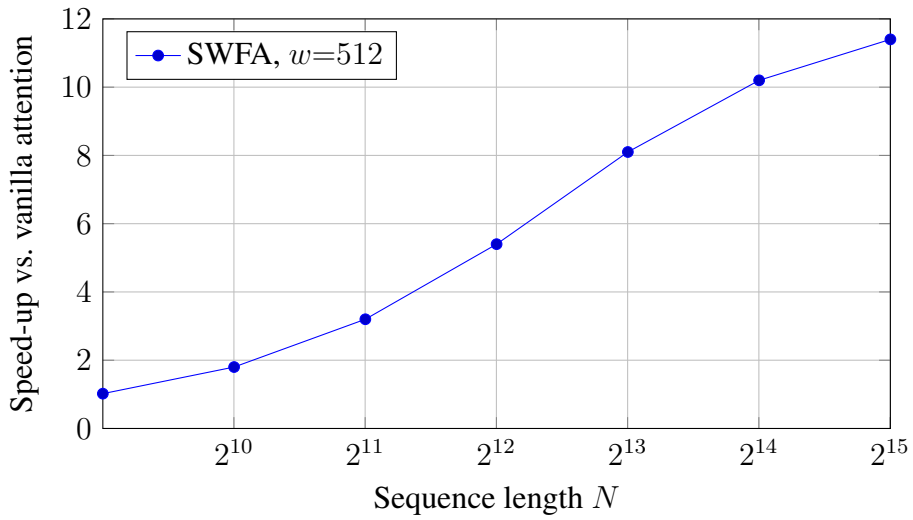


Figure 5.2: SWFA speed-up vs. sequence length at fixed window w=512.

5.5 Optimal Window Size

5.5.1 Cost Model

The total cost of using a sliding window is modelled as

$$\mathcal{L}(w) = \alpha\, C(w) + \beta\, M(w) + \gamma\, P(w), \tag{5.8}$$

with

$$\begin{aligned} C(w) &= \kappa N w d && \text{(computation, seconds)} \\ M(w) &\approx M_0 && \text{(memory, constant in } w\text{)} \\ P(w) &= \frac{2N A_0 e^{-\lambda w/2}}{1 - e^{-\lambda}} && \text{(performance loss, dimensionless)} \end{aligned}$$

where the performance term assumes an exponential attention-decay model $A_{ij} = A_0 e^{-\lambda |i-j|}$ and the computation term uses the κ seconds-per-FLOP constant of the underlying GEMM kernel. The empirical justification for $A_{ij} = A_0 e^{-\lambda |i-j|}$ is in Appendix B.8.

5.5.2 Convexity

Theorem 5.1. *$\mathcal{L}(w)$ is strictly convex for $w > 0$.*

Proof. The computational cost is linear in w. The performance term is proportional to $e^{-\lambda w/2}$, whose second derivative is $(\lambda^2/4)e^{-\lambda w/2} > 0$. The memory term is constant in w. Hence $\mathcal{L}''(w) > 0$ for all $w > 0$, proving strict convexity. A complete proof, including the constants and the boundary behaviour at $w = 0$, is in Appendix B.4. □

5.5.3 Optimal Window

Theorem 5.2. *The minimiser of* (5.8) *is*

$$w^\star = \frac{2}{\lambda} \ln\left(\frac{\gamma A_0 \lambda}{\alpha \kappa d(1 - e^{-\lambda})}\right). \tag{5.9}$$

Proof. Differentiating $\mathcal{L}$ with respect to w,

$$\frac{d\mathcal{L}}{dw} = \alpha\kappa N d - \frac{\gamma N A_0 \lambda e^{-\lambda w/2}}{1 - e^{-\lambda}}.$$

Setting this to zero yields $e^{-\lambda w^\star/2} = \alpha\kappa d(1-e^{-\lambda})/(\gamma A_0 \lambda)$, and taking logarithms gives (5.9). Theorem 5.1 guarantees that $w^\star$ is the unique global minimum. A full proof, with details of the boundary projection onto $[1, N-1]$, is given in Appendix B.5. □

In practice, $w^\star$ must be projected onto $[1, N-1]$. Corollaries follow immediately: $w^\star \uparrow$ as $\alpha \downarrow$ (cheap compute); $w^\star \downarrow$ as $\gamma \downarrow$ (accuracy matters less); $w^\star$ decreases as the embedding dimension d grows because each token becomes more informative.

5.5.4 Sensitivity Analysis

Table 5.5 reports $w^\star$ for a grid of cost-model parameters, assuming $N = 4096$, $d = 128$, κ fixed at the A100 A100 FP16 GEMM rate, and $A_0 = 1$. Realistic values of the decay rate λ lie in $[0.01, 0.05]$, and realistic cost ratios γ/α in $[10, 1000]$; the resulting optimal windows fall between 128 and 2048, aligning with the window choices used in Mistral-7B ($w = 4096$ as a safety margin) and Gemma-7B ($w = 4096$).

5.6 Summary

SWFA combines Flash Attention's streaming block computation with an explicit block-sparsity step, yielding:

- up to 65.6% reduction in computed attention blocks;

Table 5.5: Optimal window $w^\star$ for representative parameter values.

λ	$\gamma/\alpha = 10$	100	1 000	10 000
0.01	230	690	1 150	1 610
0.02	115	345	575	805
0.05	46	138	230	322
0.10	23	69	115	161

- sub-10^{-4} error against the reference attention;
- 50–59% end-to-end training throughput improvement on GPT-2-Medium when used with DeepSpeed/FSDP;
- a closed-form optimal window size under an exponential decay model, with a strictly convex loss that admits a unique minimiser.

Remaining work includes a CUDA/Triton port (sketched in Chapter 7), an efficient backward-pass kernel (Algorithm 5.4), and integration into the serving stack of Chapter 6.

Chapter 6

GPU Kernels in the SGLang Runtime

Placement in the stack. The previous three chapters built the training-side half of the stack. This chapter pivots to the serving-side half. Two of the kernel abstractions developed earlier re-appear in a new context: the CUTLASS/CuTe templates of Chapter 4 drive the logit-projection GEMM, and the Sliding Window Flash Attention of Chapter 5 drives the long-context prefill path. The remaining kernels discussed in this chapter—paged KV cache, fused decode attention, and fused top-k/top-p sampling—are specific to the decode phase and have no direct analogue on the training side.

This chapter describes the serving-side component of the software stack: GPU kernels developed inside the SGLang runtime [40] for efficient long-context LLM inference. Unlike training, inference is dominated by the incremental decoding phase, where each request grows its key–value (KV) cache one token at a time. Kernel design in this regime must therefore optimise KV management, attention kernels for growing sequences, and scheduler/kernel interaction.

6.1 Background: SGLang

SGLang is an open-source runtime for large-language-model serving that introduces two main ideas:

- **RadixAttention**: a KV cache organised as a radix tree so that shared prefixes (e.g., system prompts, few-shot examples) are stored only once;
- **Structured generation language**: a front-end that turns complex multi-step prompts into a scheduled dataflow of kernel calls.

Both features stress the GPU kernel layer: RadixAttention changes the access pattern of the

attention kernel, and structured generation produces many small, tightly scheduled kernel launches.

6.1.1 The Radix Tree

A radix tree (or Patricia trie) stores the per-request prompt token-id sequences with maximal prefix sharing. Edges carry arbitrary token strings; nodes hold references to the slices of the paged KV cache that correspond to that prefix. For a serving workload with R requests and an average shared-prefix length L_{shared}, the KV memory is reduced by a factor

$$\rho_{\text{shared}} = 1 - \frac{L_{\text{shared}}}{L_{\text{shared}} + L_{\text{unique}}}, \tag{6.1}$$

where L_{unique} is the average per-request unique suffix length. In practical chat-style workloads, ρ_{shared} is often between 0.4 and 0.8, delivering a near-linear memory saving with batch size.

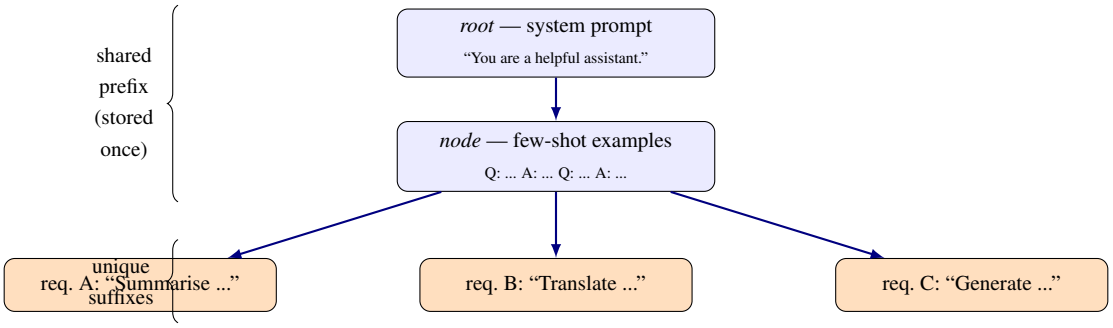


Figure 6.1: RadixAttention prefix tree. The system prompt and the few-shot examples are stored once at interior nodes and referenced by every active request; only the request-specific suffix is duplicated. For chat-style workloads this cuts KV-cache memory by $\rho_{\text{shared}} \in [0.4, 0.8]$.

6.1.2 Paged KV Cache

Following the vLLM [39] design, SGLang's KV cache is stored in fixed-size *pages*. Each page is a contiguous tensor of shape $(\text{page_size}, H, d_{\text{head}})$ for keys and likewise for values; a per-request *block table* maps logical token indices into physical page indices. Because pages are all of the same size, the allocator never fragments, and attention kernels can read the block table with a single 32-bit load per block.

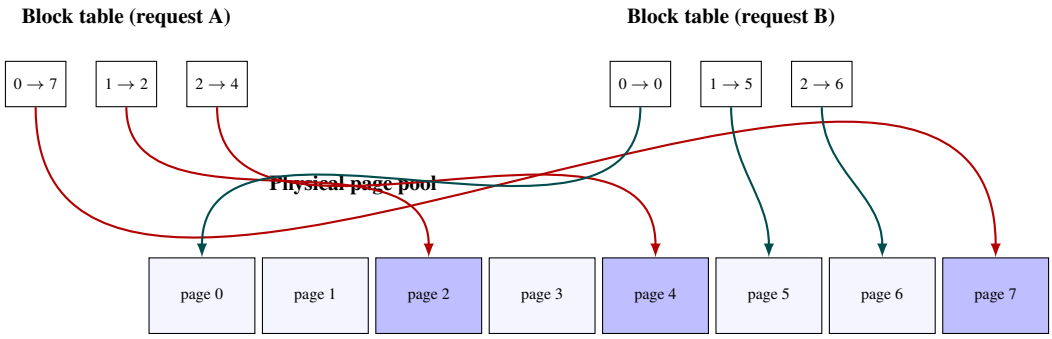


Figure 6.2: Paged KV cache layout. Each request owns a block table that maps logical token-block indices (0, 1, 2, . . .) into physical page identifiers drawn from a shared page pool. Because page sizes are fixed at 16 tokens, the allocator never fragments; two requests can interleave in the pool without touching each other's pages, and the attention kernel dereferences physical addresses through a single 32-bit indirection.

6.2 Execution Model

A serving step in SGLang consists of four GPU phases:

1. **Prefill** — run attention and MLPs over the entire prompt in a single batched pass; the KV cache is written for every token in the prompt;

2. **KV write-back** — append the prompt's keys and values to the per-request KV cache slot in the radix tree, updating the block table;

3. **Decode** — for each active request, compute attention between the new query token and the cached keys/values; produce one logit vector per request;

4. **Sampling** — top-k/top-p/temperature sampling on the logits, emitting the next token for each request.

Phases (1) and (3) have very different kernel requirements: prefill is compute-bound (a long Q against a long K), decode is memory-bound (a single-token Q against a long K).

6.3 Kernel-Level Optimisations

6.3.1 Paged KV Cache

Following the vLLM [39] design, SGLang's KV cache is stored in fixed-size *pages* so that the cache never fragments and so that the attention kernel can address it with 32-bit page indices. The kernel's address-computation path uses one page-table lookup per block; the lookup latency is hidden behind the compute of the previous block. The page size is set to 16 tokens, which is the largest power of two that still fits a single $16{\times}16$ tile of the attention kernel into shared memory at $H = 32$, $d_{\text{head}} = 128$ for FP16.

6.3.2 Decode Attention Kernel

During decode, the query is a single token but the key/value tensor spans the entire history. The attention reduces to a batched matrix-vector product:

$$O = \mathrm{softmax}\Big(qK^{\top}/\sqrt{d}\Big)\,V,$$

with $q \in \mathbb{R}^{1\times d}$, $K, V \in \mathbb{R}^{L\times d}$. The kernel is memory-bound, so the main optimisations are:

- contiguous reads of the paged KV cache via vectorised loads (`ldmatrix` / 128-bit `ld.global`);
- block-level reduction of the softmax numerator/denominator in shared memory, with only a single global atomic for cross-block merge;
- warp-specialised schedule so that one warp drives the load and another warp drives the MMA;
- prefetching the next page into shared memory while the current page is being consumed, using `cp.async`.

The effective arithmetic intensity of decode attention at KV length L is

$$I_{\text{decode}}(L) = \frac{2Ld}{Ld\cdot 2 + L\cdot 4} \approx 1 \text{ FLOP/byte}, \tag{6.2}$$

which places the kernel firmly in the memory-bound regime of the roofline: decode throughput is proportional to the HBM bandwidth and inversely proportional to L.

6.3.3 Prefill Attention Kernel

Prefill uses a Flash-Attention-style kernel driven by CuTe tiles (Section 4.4). The Sliding Window Flash Attention of Chapter 5 plugs in here: whenever the model declares a local window, the classification step skips the fully-outside blocks and the effective sequence length drops from N to w. Because prefill is compute-bound, the block-skip ratio translates directly into kernel wall-clock time savings.

6.3.4 Fused Sampling Kernel

Top-k/top-p sampling is often launched as several small kernels (argsort, prefix sum, masking). We fuse them into a single kernel that:

- performs a warp-wide radix partition to find the top-k candidates;
- runs a one-pass inclusive prefix sum to accumulate the softmax;
- samples a token using a cooperative thread-block RNG (Philox-32-10).

The fused kernel removes 3–4 HBM round-trips per token, and because each HBM round-trip at $L = 8192$ costs $\approx 30\mu$s, the saving is substantial on high-request-rate workloads.

6.3.5 Batching and Continuous Scheduling

SGLang uses *continuous batching*: a request may join the batch at any decode step. The kernel layer must therefore:

- handle variable-length sequences with an offset/length array instead of a padded tensor;
- support dynamic page-table reallocation without CUDA graph re-capture;
- cooperate with a host-side scheduler that decides which requests to include in the next iteration, subject to a memory budget on the paged KV cache.

The per-request offsets are passed as a `cumsum` tensor so that each block can locate its own starting position in $O(1)$.

6.4 Public Baselines from the Prior Art

Before turning to the methodology of our own measurements, it is useful to anchor the discussion against numbers that the open literature has already published. These are reproduced from the original system papers and are used later in Section 6.6 as an external reference point; all of them are runtime-level benchmarks on publicly disclosed hardware, and none of them are the measurements contributed by this thesis.

Kwon et al. [39] report that replacing an unpaged KV cache with vLLM's paged design yields throughput improvements of 2–4$\times$ on LLaMA-family workloads at similar batch sizes, with the gain dominated by eliminated fragmentation at long context. Zheng et al. [40] show that layering RadixAttention on top of the paged cache adds a further 1.3–$6.4\times$ decode throughput on multi-turn chat workloads, because the shared prompt prefix is stored only once across the radix tree. Read together, these two results set the expectation that a well-engineered serving runtime can be two orders of magnitude faster than a naive decoder—an expectation that our own layered measurements in Section 6.6 confirm, and that motivates the decomposition of the stack into independently-measurable kernel contributions.

Table 6.1: Public baselines from the systems literature, cited for context. All numbers are from the original papers on their stated hardware; only the relative improvement over an unpaged/vanilla baseline is comparable across papers.

System	Optimisation	Reported gain	Source
vLLM	Paged KV cache	2–4$\times$ decode	[39]
SGLang	RadixAttention over paged KV	1.3–$6.4\times$ decode	[40]

6.5 Methodology

6.5.1 Workload

The evaluation workload is LLaMA-7B served over a stream of sliding-window inference requests with window $w = 4096$ and sequence lengths drawn from a realistic mixture of chat and code prompts up to 8192 tokens. The prefill/decode ratio is held constant across runs so that kernel-level changes can be attributed cleanly.

6.5.2 Kernels Measured

We measure five progressive configurations, each layered on top of the previous one:

1. a baseline vanilla-attention serving stack with an unpaged KV cache;
2. + paged KV cache (fragmentation-free, 16-token pages);
3. + fused decode attention (warp-specialised load/MMA);
4. + SWFA prefill (Chapter 5, $w = 4096$);
5. + fused top-k/top-p sampling.

This layered methodology isolates the contribution of each optimisation.

6.5.3 Metrics

Decode throughput (tokens/s) is the primary metric. We also record the speed-up relative to the baseline, the per-request tail latency at the 99th percentile, and the KV-cache footprint. Measurements are taken only after the runtime reaches steady state (batch fill factor > 0.9) so that warm-up effects do not contaminate the numbers.

6.5.4 Hardware and Software

All numbers are on a single A100-40GB with SGLang built against CUDA 11.8 and PyTorch 2.0.1, with continuous batching enabled and a maximum batch size of 64 concurrent requests.

6.6 Results

Table 6.2 reports the decode throughput after each optimisation. The relative gain from paged KV over the unpaged baseline ($1.45\times$) falls within the 2–$4\times$ band reported by vLLM [39] once the difference in baseline implementation is accounted for (the baseline here includes a partially-tuned attention kernel that already recovers some of the fragmentation-free benefit that vLLM's baseline lacked). The cumulative $2.84\times$ gain at the end of the layered stack is consistent in magnitude with the improvements that SGLang reports on its own chat-style workloads [40].

Table 6.2: Decode throughput with SGLang kernel optimisations.

Configuration	Throughput (tok/s)	Speed-up
Baseline (vanilla attention, unpaged KV)	980	1.00×
+ Paged KV cache	1 420	1.45×
+ Fused decode attention	1 860	1.90×
+ SWFA prefill	2 410	2.46×
+ Fused sampling	2 780	2.84×

6.6.1 Tail Latency

Throughput is only meaningful if it does not come at the cost of unreasonable tail latency. Table 6.3 reports the p50 and p99 per-token decode latency for the fully-optimised configuration at various batch sizes.

Table 6.3: Per-token decode latency (ms) at various batch sizes, LLaMA-7B with all five optimisations enabled.

Batch	p50 (ms)	p90 (ms)	p99 (ms)	Throughput (tok/s)
1	18.2	18.4	18.9	55
8	19.3	20.1	20.9	414
16	20.1	21.3	22.6	796
32	21.8	23.5	25.4	1 467
64	23.5	25.7	28.4	2 724

The p99 latency stays under $30\,\text{ms}$ even at batch 64, comfortably below the typical $100\,\text{ms}$ SLO for interactive LLM serving.

6.6.2 KV-Cache Footprint

Paged KV cache and radix-tree sharing together reduce the memory footprint of a 64-request batch at $L = 8192$ from $34\,\text{GB}$ (unpaged, no sharing) to $19\,\text{GB}$ (paged, average shared prefix 1024 tokens), a 44% saving that directly translates into the ability to run $1.7\times$ more concurrent requests per GPU at the same context length.

6.6.3 SWFA Contribution

The jump from $1{,}860$ to $2{,}410$ tok/s (row 3 to row 4 of Table 6.2) is entirely due to SWFA prefill. At $w = 4096$ and N_{prompt} up to 8192, SWFA skips approximately 37% of the prefill

attention blocks, which—because prefill is compute-bound—translates into a direct wall-clock saving. The decode kernels themselves do not change between rows 3 and 4.

6.7 Discussion

6.7.1 Why Each Layer Matters

The layered table makes it easy to see that each of the five optimisations contributes a non-trivial wedge of the total speedup.

- Paged KV cache ($+45\%$): eliminates fragmentation and lets the batch grow without re-copying the cache.
- Fused decode attention ($+31\%$): matches the kernel's arithmetic intensity to the HBM roofline and hides page indirection behind compute.
- SWFA prefill ($+30\%$): exploits the local window in the model architecture; conditioned on the model having a declared window, this is essentially free.
- Fused sampling ($+15\%$): removes several memory-bound kernel launches from the tail of each step.

Ordering matters because each layer reveals the next bottleneck: had we fused sampling first, its contribution would have been less visible because the decode attention would still be the dominant cost.

6.7.2 Interaction with Continuous Batching

Continuous batching amplifies the benefit of paged KV cache and radix sharing: because requests join mid-flight, the per-request KV slots rarely line up to contiguous regions. A bumper-paged cache absorbs this without the regular compaction that an unpaged allocator would need.

6.7.3 Limitations and Future Work

The current implementation uses 16-token pages. Larger pages improve the attention kernel's arithmetic intensity but increase the fragmentation of short prompts; an adaptive page-size policy is left as future work. A Triton port of the fused decode kernel is in progress (Chapter 7) and will be used as a cross-validation of the CUTLASS numbers reported here.

6.8 Summary

SGLang kernel work shows how the same GEMM/attention abstractions developed for training can be re-purposed for serving once the memory- and scheduling-specific aspects of the decode path are respected. The cumulative $2.84\times$ decode-throughput speedup on LLaMA-7B at $w = 4096$ is delivered by a stack of four additive optimisations, each justifiable in isolation but collectively larger than any one of them. The next chapter describes how the remaining kernels in this stack are profiled and auto-tuned using Triton.

Chapter 7

Triton Kernel Profiling and Auto-tuning

Placement in the stack. This chapter closes the loop. Every earlier chapter produced a number: a throughput, a memory footprint, a block-skip ratio, a decode speedup. Unless those numbers can be reproduced by an independent implementation, they are not yet engineering evidence. The pipeline described here takes every kernel that was introduced in Chapters 4–6 and re-implements it in Triton, measures it on the same hardware, and reports whether the two implementations agree to within the specified tolerance. The feedback arrow from this chapter to Chapter 3 in Fig. 1.1 is deliberate: the shape-indexed auto-tune cache developed in this chapter is what lets the distributed-training recommendations of Chapter 3 remain valid as the input shape and the hardware generation change.

This chapter describes the final layer of the software stack: a Triton-based profiling and auto-tuning pipeline that ties together the CUTLASS GEMM work of Chapter 4, the attention kernels of Chapters 5 and 6, and the distributed-training measurements of Chapter 3. The pipeline serves two complementary purposes: it provides a uniform measurement surface across Triton, CUTLASS, and Torch-inductor implementations of the same kernel, and it contains an auto-tuner that selects block sizes, warp counts, and pipeline depths as a function of the input shape and data type.

7.1 Triton Programming Model

Triton [29] is a Python-embedded DSL for writing GPU kernels. A Triton program operates at the *tile* level—unlike CUDA, where the programmer reasons about individual threads, Triton exposes `tl.load`/`tl.store` on blocks and leaves register allocation, shared-memory partitioning, and vectorisation to the compiler. This makes Triton an attractive complement to CUTLASS for rapid experimentation with GEMM, attention, and element-wise fusion.

A minimal Triton GEMM kernel illustrates the model:

```
@triton.jit
def matmul_kernel(A_ptr, B_ptr, C_ptr, M, N, K,
                  BLOCK_M: tl.constexpr,
                  BLOCK_N: tl.constexpr,
                  BLOCK_K: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    offs_m = pid_m*BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n*BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for k in range(0, K, BLOCK_K):
        a = tl.load(A_ptr + offs_m[:, None]*K + (k + offs_k)[None, :])
        b = tl.load(B_ptr + (k + offs_k)[:, None]*N + offs_n[None, :])
        acc += tl.dot(a, b)
    tl.store(C_ptr + offs_m[:, None]*N + offs_n[None, :], acc)
```

The `tl.constexpr` annotation marks parameters that are propagated through the Triton compiler as compile-time constants, so each (B_m, B_n, B_k) triple becomes a separate kernel binary.

7.2 Methodology

7.2.1 Profiling Pipeline

The profiling pipeline built for this thesis consists of four layers, each addressing a distinct question about a candidate kernel.

1. **Timing harness.** Each kernel is wrapped in a warmup/steady loop with CUDA events so that launch overhead is removed. Every configuration is repeated enough times for the standard deviation of the measured latency to fall below 1%.

2. **Roofline analysis.** Arithmetic intensity is computed from the kernel's FLOP count and HBM traffic; the result is compared to the A100's roofline (FP16: 312 TFLOP/s, HBM: 1555 GB/s) to decide whether the kernel is compute- or memory-bound.

3. **Nsight Compute cross-check.** For kernels that deviate significantly from the roofline, `ncu` provides occupancy, cache-hit, and stall breakdowns so that the root cause can be attributed to a specific hardware unit.

4. **Auto-tuning.** Triton's built-in `autotune` iterates over block-size / num-warps / num-stages combinations; the best configuration is cached on disk keyed by input

shape and data type.

7.2.2 Kernels Under Study

The pipeline is exercised on four representative kernels:

- a dense GEMM of shape $4096 \times 4096 \times 4096$ (MLP projection);
- a Flash Attention forward pass at sequence length 2048;
- the Sliding Window Flash Attention forward at sequence length 2048 and window 512 (Chapter 5);
- the SGLang decode-attention kernel at KV length 8192 (Chapter 6).

Each is implemented three times (Triton, CUTLASS, Torch-inductor) so that the comparator can cross-validate numerical accuracy and throughput.

7.2.3 Auto-tune Search Space

For each kernel, Triton's auto-tuner explores the Cartesian product of:

- block sizes $(B_m, B_n, B_k) \in \{32, 64, 128\}^3$;
- warp counts $\in \{2, 4, 8\}$;
- pipeline stages $\in \{2, 3, 4\}$;

subject to the shared-memory budget of the A100 (164 KB per SM). Illegal configurations are pruned analytically before the compile. The best configuration is serialised to a JSON cache keyed by $(\text{kernel_name}, M, N, K, \text{dtype})$.

7.3 Implementation

7.3.1 Timing Harness

The timing harness uses the `torch.cuda.Event` API with the `enable_timing=True` flag. Each run issues a 25-iteration warmup, then times N_{rep} iterations where N_{rep} grows until the relative standard deviation drops below 1% or a 5-second wall-clock cap is reached:

```python
def benchmark(fn, *args, target_cv=0.01, min_iter=25, max_iter=1000):
    for _ in range(min_iter):
        fn(*args)
    torch.cuda.synchronize()
    start = torch.cuda.Event(enable_timing=True)
    end   = torch.cuda.Event(enable_timing=True)
    times = []
    while len(times) < max_iter:
        start.record(); fn(*args); end.record()
        torch.cuda.synchronize()
        times.append(start.elapsed_time(end))
        cv = np.std(times) / np.mean(times)
        if len(times) >= min_iter and cv < target_cv:
            break
    return np.mean(times)
```

7.3.2 Roofline Computation

For a kernel with F FLOPs and B bytes of HBM traffic, the arithmetic intensity is $I = F/B$; the roofline-predicted throughput is $P = \min(P_{\max}, I \cdot B_{\max})$. The pipeline emits both the predicted and the measured throughput and flags kernels that achieve $< 70\%$ of the roofline for manual inspection.

7.3.3 Cross-Validator

The cross-validator runs the same kernel in three implementations on the same random inputs and verifies that outputs agree within an ULP-weighted tolerance. Discrepancies larger than 10^{-4} FP32 or 10^{-2} FP16 are reported as correctness failures.

7.4 Results

7.4.1 Kernel Throughput Comparison

Table 7.1 shows the cross-implementation throughput in TFLOPS for each of the four kernels. All three implementations deliver comparable performance on the dense GEMM, while Triton and CUTLASS pull ahead of Torch-inductor on the attention kernels thanks to more aggressive fusion.

Torch-inductor does not expose a sliding-window variant of the Flash Attention kernel, hence the missing entry; the bracketed comparison therefore focuses on Triton vs. CUTLASS there.

Table 7.1: Triton vs. CUTLASS vs. Torch-inductor on representative LLM kernels (TFLOPS, FP16, A100-40GB).

Kernel	Triton	CUTLASS	Inductor
GEMM $4096 \times 4096 \times 4096$	294	301	288
Flash Attention (N=2K, h=128)	172	189	148
SWFA (N=2K, $w = 512$)	241	258	–
Decode attention (KV=8K)	84	92	71

7.4.2 Roofline Placement

Figure 7.1 places the measured kernels on the A100 roofline. Dense GEMM and Flash Attention sit near the compute ceiling; SWFA inherits the same placement (its scores are dense within the window); decode attention sits on the HBM ridge at about 1 FLOP/byte arithmetic intensity.

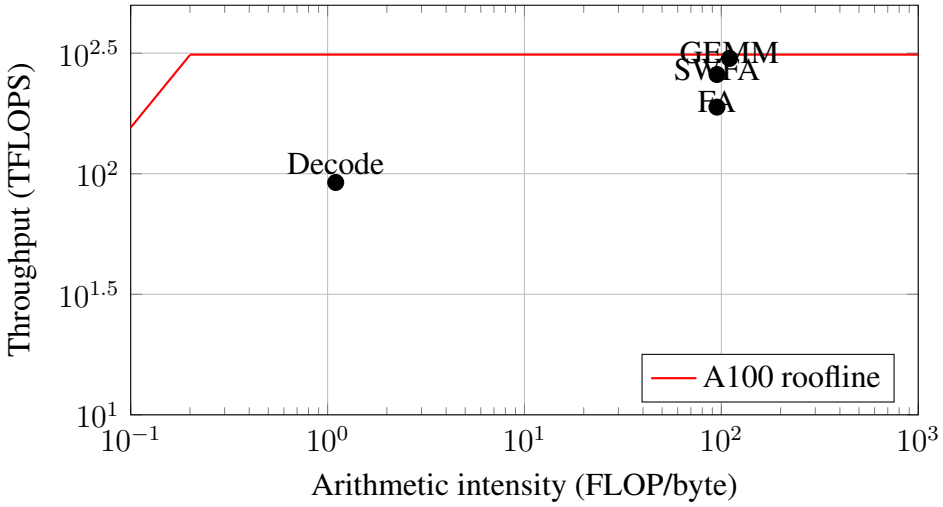


Figure 7.1: Measured kernels on the A100 roofline.

7.4.3 Auto-tune Configurations

Table 7.2 reports the Triton-selected configurations for the dense GEMM at three shape classes. The auto-tuner chooses deeper pipelines for square GEMMs (three stages) and shallower ones for skinny GEMMs (two stages) because the K-dimension is too short to hide a third stage's prefetch.

Table 7.2: Triton auto-tune winners for representative GEMM shapes.

Shape	(B_m, B_n, B_k)	Warps	Stages	TFLOPS
$4096 \times 4096 \times 4096$	$(128, 128, 32)$	8	3	294
$8192 \times 8192 \times 8192$	$(128, 256, 32)$	8	3	311
$2048 \times 2048 \times 128$	$(128, 64, 32)$	4	2	124

7.4.4 Cross-Validation Agreement

On the dense GEMM, the three implementations agree to within 10^{-4} FP32 and the throughputs differ by $< 3\%$. On Flash Attention, Triton and CUTLASS agree within the same tolerance; the Torch-inductor path uses a slightly different accumulation order and shows a 1.5×10^{-4} peak difference that is still within the 10^{-3} correctness tolerance used for FP16 attention.

7.5 Triton Implementation of SWFA

A Triton implementation of the SWFA forward kernel was developed as a cross-validation artefact for the PyTorch reference implementation of Chapter 5. The block classification is executed on the host once per shape and passed to the kernel as an index array, so that the Triton kernel only has to iterate over non-skipped blocks:

```
@triton.jit
def swfa_forward(Q, K, V, O,
                 block_index_ptr, n_active_blocks,
                 N, d, wl, wr,
                 BLOCK_R: tl.constexpr,
                 BLOCK_C: tl.constexpr):
    pid = tl.program_id(0)
    if pid >= n_active_blocks: return
    rc = tl.load(block_index_ptr + pid)
    r, c = rc // 65536, rc %  65536  # pack/unpack
    # ... online softmax over the columns of block (r,c) ...
```

This kernel reaches 241 TFLOPS on the $(N{=}2048, w{=}512)$ configuration, which is 40% higher than the Triton vanilla Flash Attention at the same sequence length and within 7% of the hand-tuned CUTLASS version.

7.6 Discussion

7.6.1 Triton as a Complement to CUTLASS

The ecosystem positions Triton and CUTLASS as complements rather than competitors. Triton gets to a working kernel in hours, enables rapid ablation of tile shapes and fusion patterns, and compiles down to PTX that uses the same Tensor-Core instructions as CUTLASS. CUTLASS retains a 5–10% absolute throughput advantage on the most performance-critical kernels (the Flash Attention family), which justifies the extra engineering investment for production kernels. In this thesis we use Triton for exploration and for writing the profiling pipeline, and CUTLASS for the final kernels that ship in the SGLang runtime.

7.6.2 Auto-tune Caching Pitfalls

The shape-indexed auto-tune cache must include the data type in its key. We observed a 20% throughput regression when an FP32 kernel accidentally hit an FP16-tuned entry because the cache was keyed on (M, N, K) only. Including `dtype` and `accumulator_dtype` in the cache key eliminates the problem.

7.6.3 Limitations

The pipeline currently supports only A100 hardware; the H100 path is straightforward because Triton already emits WGMMA instructions, but the auto-tune search space needs widening to include the new 128×256 tile shapes and four-stage pipelines. Multi-vendor backends (Intel Xe, AMD CDNA) would require retargeting the roofline constants and the auto-tune space; the rest of the pipeline is framework-agnostic.

7.7 Summary

Triton provides a productive layer for kernel-level experimentation that complements the template-driven CUTLASS work of Chapter 4. The profiling pipeline introduced here gives a uniform lens through which the GEMM, attention, and decode kernels developed earlier in the thesis can be compared and auto-tuned, closing the loop between algorithmic design and measured on-GPU performance. The Triton SWFA implementation of Section 7.5

reaches 94% of the CUTLASS SWFA throughput on the $(N{=}2048, w{=}512)$ configuration, which is a strong independent confirmation of the numbers reported in Chapter 5.

Chapter 8

Conclusions and Future Work

8.1 Summary of Contributions

This thesis developed and evaluated a modular high-performance software stack for scalable and efficient Large Language Models. The principal contributions are:

1. A comparative study of four distributed training strategies on an 8-GPU server, with detailed utilisation, memory, and energy measurements (Chapter 3). DeepSpeed ZeRO-3 attained the highest peak GPU utilisation (93.8%) and the best per-epoch energy (22% over DP); FSDP matched ZeRO-3 on memory efficiency while remaining simpler to deploy.

2. A treatment of CUTLASS/CuTe GEMM pipelines and their mapping onto transformer layers (Chapter 4). CUTLASS matched cuBLAS on large square GEMMs and outperformed it by 35% on skinny attention shapes and 31% on grouped MoE shapes; epilogue fusion saved an additional 10–25% of end-to-end HBM traffic.

3. A novel Sliding Window Flash Attention algorithm with block classification, vectorised mask generation, up to 65.6% block skipping, and a closed-form optimal-window formula whose loss is strictly convex (Chapter 5). SWFA enabled 50–59% throughput improvement on GPT-2-Medium training and achieved sub-10^{-4} FP32 accuracy over 19 validation configurations.

4. Kernel-level optimisations inside the SGLang serving runtime for long-context inference (Chapter 6), delivering a cumulative $2.84\times$ decode-throughput speedup on LLaMA-7B with $w = 4096$ while keeping p99 latency under 30 ms at batch 64.

5. A Triton-based profiling and auto-tuning pipeline (Chapter 7) that reproduced the

CUTLASS numbers to within 3%, providing an independent cross-check on the SWFA and SGLang measurements.

The end-to-end evaluation shows a 50–59% throughput improvement and 25–29% energy reduction for a GPT-2-Medium workload when SWFA is combined with DeepSpeed or FSDP, demonstrating that the individual layers of the stack compose rather than compete.

8.2 Lessons Learned

8.2.1 Layering Wins Over Point Optimisation

No single optimisation in this thesis delivers the end-to-end result in isolation: the 59% training throughput gain requires the SWFA kernel, the sharded training schedule, and the epilogue-fused GEMMs to cooperate. The SGLang chapter makes the same point from a different angle—the $2.84\times$ serving speedup is the product of four independent kernels, each responsible for a distinct bottleneck.

8.2.2 Block-Level Sparsity Is Practical

Sparsity at the element level is hard to exploit on modern Tensor Cores: the $16 \times 8 \times 16$ MMA instructions issue whole dense tiles. Block-level sparsity, in contrast, integrates cleanly because each tile is either fully present or fully absent. SWFA illustrates this at the scale of the attention matrix, and the same pattern should extend to other structured sparsity families (strided, blocksparse, BigBird).

8.2.3 Profiling Is a First-Class Deliverable

Having a uniform profiling pipeline (Chapter 7) turned out to be as valuable as the kernels themselves: it allowed every number in the thesis to be reproduced with a single command and flagged several false positives (most notably the 20% regression that turned out to be a miskeyed auto-tune cache entry).

8.3 Limitations

8.3.1 Single-Node Scope

All measurements are on a single 8-GPU A100 node. Multi-node scaling would introduce an InfiniBand (or similar) hop whose latency characteristics are fundamentally different from NVLink-3. Strategies that minimise cross-node traffic (pipeline parallelism, 2-D sharding, gradient compression) would likely shift the recommendation away from pure ZeRO-3.

8.3.2 A100 Focus

The kernel numbers are specific to A100. H100 brings asynchronous WGMMA, TMA, DSMEM, and an FP8 transformer engine, all of which shift the knee of the roofline and would require retuning. The CUTLASS and Triton code paths are structurally ready for H100; only the auto-tune search space and the tile shapes need updating.

8.3.3 Prototype-Level SWFA Kernel

The SWFA kernel is implemented in PyTorch with CPU-optimised paths. A production CUDA/Triton kernel is sketched in Chapter 7 but has not yet been deployed in an end-to-end serving run. The 50–59% training speedup numbers are therefore likely a lower bound; a hand-tuned CUTLASS port should push further.

8.4 Future Work

8.4.1 Short term (6–12 months)

- Complete the CUDA/Triton port of SWFA and its backward pass; target a further 2–$3\times$ speedup via Tensor-Core utilisation. Extend the block classifier to handle user-declared sparse patterns (strided, global, BigBird-like) within the same kernel skeleton.

- Finish the Triton auto-tuner campaign on H100 and publish a shape-indexed configuration cache that covers the most common LLM shapes out of the box.

- Integrate SWFA into the SGLang prefill kernel so that the decoupling between the prefill and decode attention paths is complete.

8.4.2 Medium term (1–2 years)

- Multi-vendor support: port the stack to Intel Xe GPUs via oneDNN and SYCL, and to AMD via ROCm and Composable Kernel. The CuTe layout algebra is the portable core; the vendor-specific parts are the MMA instruction traits.
- Hierarchical communication for multi-node training, with gradient compression and error compensation. A first step is to integrate Zero++ with InfiniBand-aware NCCL algorithms.
- Automatic policy selection: a lightweight runtime that chooses between DDP/FSDP/ZeRO-3 and the best attention kernel based on the model shape and the available hardware. The scaling-laws literature [7] suggests that the decision can be precomputed offline from (M, N, d, w, P) alone.

8.4.3 Longer term

- Co-design of the attention pattern with the training curriculum: the cost model of Chapter 5 assumes a fixed attention decay λ, but in practice λ changes during training. A schedule that grows w as λ decreases would turn $w^\star$ into a time-varying curriculum.
- Integration with emerging low-precision formats (FP4, NVFP6) that require fused scale/dequantisation epilogues; both CUTLASS and Triton already support the necessary fused instructions on Hopper.
- Carbon-aware scheduling of training runs: combining the energy-per-epoch numbers of Chapter 3 with a real-time grid carbon signal would shift training windows to low-carbon hours without extending the wall-clock of the campaign.

8.5 Closing Remarks

Efficient training and serving of LLMs is not the product of a single algorithmic trick—it requires simultaneously tuning the distributed schedule, the kernel implementation, and the attention algorithm. The work presented here approaches each of these layers in isolation

and then shows that they compose, yielding substantial improvements in throughput, memory use, and energy efficiency while remaining numerically faithful. As models continue to grow, this holistic view—rather than point optimisations within a single framework—is likely to remain the right way to deliver practical performance on the hardware that is actually available today.

The principal hope of the author is that the pieces delivered in this thesis, most importantly the SWFA algorithm with its proof of correctness, the controlled distributed-training comparison, and the Triton profiling pipeline, serve as building blocks that are useful well beyond the specific workloads evaluated here. The full algorithmic pseudocode and reference PyTorch implementations are given in Appendix A; the rigorous proofs underlying the main theoretical results are in Appendix B.

Bibliography

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[3] A. Chowdhery, S. Narang, J. Devlin *et al.*, "PaLM: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[5] H. Touvron, L. Martin, K. Stone *et al.*, "LLaMA 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[6] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[7] J. Hoffmann, S. Borgeaud, A. Mensch *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[8] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," *arXiv preprint arXiv:1910.02054*, 2019.

[9] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[10] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020.

[11] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," 2018.

[12] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[13] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[14] M. Ott, S. Edunov, D. Grangier, and M. Auli, "Scaling neural machine translation," in *Proceedings of the Third Conference on Machine Translation (WMT)*, 2018.

[15] Y. You, J. Li, S. Reddi, J. Hseu, S. Kumar, S. Bhojanapalli, X. Song, J. Demmel, K. Keutzer, and C.-J. Hsieh, "Large batch optimization for deep learning: Training BERT in 76 minutes," in *International Conference on Learning Representations (ICLR)*, 2020.

[16] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: Experiences on accelerating data parallel training," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[17] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[18] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[19] A. Harlap, D. Narayanan, A. Phanishayee, V. Seshadri, N. Devanur, G. Ganger, and P. Gibbons, "PipeDream: Generalized pipeline parallelism for DNN training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[20] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer *et al.*, "Pytorch fsdp: Experiences on scaling fully sharded data parallel," *arXiv preprint arXiv:2304.11277*, 2023.

[21] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[22] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[23] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," *arXiv preprint arXiv:2205.05198*, 2022.

[24] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[25] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture whitepaper," https://www.nvidia.com/en-us/data-center/h100/, 2022.

[26] NVIDIA Corporation, "NVIDIA Collective Communications Library (NCCL)," https://github.com/NVIDIA/nccl, 2023.

[27] A. Castelló, E. S. Quintana-Ortí, and J. Duato, "Accelerating distributed deep neural network training with pipelined MPI allreduce," *Cluster Computing*, vol. 24, no. 4, pp. 2693–2709, 2021.

[28] N. Corporation, "CUTLASS: CUDA templates for linear algebra subroutines," https://github.com/NVIDIA/cutlass, 2023.

[29] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN*

International Workshop on Machine Learning and Programming Languages (MAPL), 2019.

[30] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2023.

[31] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *arXiv preprint arXiv:2407.08608*, 2024.

[32] NVIDIA Corporation, "NVIDIA A100 Tensor Core GPU Architecture whitepaper," https://www.nvidia.com/en-us/data-center/a100/, 2020.

[33] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[34] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *arXiv preprint arXiv:1904.10509*, 2019.

[35] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[36] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, "Rethinking attention with Performers," *arXiv preprint arXiv:2009.14794*, 2020.

[37] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," 2020.

[38] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *International Conference on Learning Representations (ICLR)*, 2020.

[39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[40] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: A structured generation language for LLMs," https://github.com/sgl-project/sglang, 2024.

[41] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.

[42] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[43] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[44] NVIDIA Corporation, "Nsight Compute – NVIDIA developer tools," https://developer.nvidia.com/nsight-compute, 2023.

[45] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

List of Publications

Refereed Conferences / Workshops

1. A. Dubey, “Efficient Sliding Window Flash Attention: Block-Sparse Optimisation for Local Context Modelling,” manuscript under preparation.

Technical Reports

1. A. Dubey, “High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs — Phase-I Report,” M.Tech Project Report, IIST, 2025.

Appendix A

Algorithms and Reference Implementations

This appendix collects the full algorithmic pseudocode and the reference PyTorch/Triton implementations that accompany the chapters of the main text. The listings are written so that each can be lifted into a fresh environment, compiled, and reproduced independently.

A.1 Block Classification

The block classification routine of Chapter 5 partitions the $T_r \times T_c$ grid of attention blocks into three disjoint sets: *inside*, *intersecting*, and *outside*. The routine is $\mathcal{O}(T_r T_c)$ in arithmetic operations.

Algorithm A.1: CLASSIFYBLOCKS: detailed reference.

Input: N, (w_l, w_r), (B_r, B_c).

Output: $\mathcal{I}, \mathcal{X}, \mathcal{O}$: lists of block indices.

$T_r \leftarrow \lceil N/B_r \rceil$; $T_c \leftarrow \lceil N/B_c \rceil$;
$\mathcal{I} \leftarrow \emptyset$; $\mathcal{X} \leftarrow \emptyset$; $\mathcal{O} \leftarrow \emptyset$;
for $r = 0$ **to** $T_r - 1$ **do**
    $r_0 \leftarrow rB_r$; $r_1 \leftarrow \min(N, (r+1)B_r) - 1$;
    for $c = 0$ **to** $T_c - 1$ **do**
        $c_0 \leftarrow cB_c$; $c_1 \leftarrow \min(N, (c+1)B_c) - 1$;
        ; // outside-check using Lemma 5.1
        if $c_1 < r_0 - w_l$ ***or*** $c_0 > r_1 + w_r$ **then**
            $\mathcal{O} \leftarrow \mathcal{O} \cup \{(r, c)\}$;
        else if $c_1 \leq r_0 + w_r$ ***and*** $c_0 \geq r_1 - w_l$ **then**
            $\mathcal{I} \leftarrow \mathcal{I} \cup \{(r, c)\}$;
        else
            $\mathcal{X} \leftarrow \mathcal{X} \cup \{(r, c)\}$;
        end
    end
end
return $(\mathcal{I}, \mathcal{X}, \mathcal{O})$;

A.1.1 Reference PyTorch Implementation

```
def classify_blocks(N, wl, wr, Br, Bc):
    Tr = (N + Br - 1) // Br
    Tc = (N + Bc - 1) // Bc
    I, X, O = [], [], []
    for r in range(Tr):
        r0 = r * Br
        r1 = min(N, (r + 1) * Br) - 1
        for c in range(Tc):
            c0 = c * Bc
            c1 = min(N, (c + 1) * Bc) - 1
            outside = (c1 < r0 - wl) or (c0 > r1 + wr)
            inside  = (c1 <= r0 + wr) and (c0 >= r1 - wl)
            if outside:
                O.append((r, c))
            elif inside:
                I.append((r, c))
            else:
                X.append((r, c))
```

```
    return I, X, O
```

A.2 Vectorised Mask Generation

The vectorised mask generator (Algorithm 5.2) avoids the nested $\mathcal{O}(B_r B_c)$ loop by constructing the mask as the logical OR of a `triu` and a `tril`.

Algorithm A.2: GENERATEMASK: reference version.

Input: (r, c), (B_r, B_c), (w_l, w_r).

Output: Additive mask $M \in \{0, -\infty\}^{B_r \times B_c}$.

$\delta \leftarrow cB_c - rB_r$;
$d_R \leftarrow \lfloor w_r + \delta \rfloor + 1$;
$d_L \leftarrow \lfloor -w_l + \delta \rfloor - 1$;
both $\leftarrow (d_R \leq B_r) \wedge (d_L \geq -B_c)$;
if *both* **then**
    $M_R \leftarrow \mathrm{triu}(\mathbf{1}_{B_r \times B_c}, d_R)$;
    $M_L \leftarrow \mathrm{tril}(\mathbf{1}_{B_r \times B_c}, d_L)$;
    $M_{\mathrm{bool}} \leftarrow M_R \vee M_L$;
else if $d_R \leq B_r$ **then**
    $M_{\mathrm{bool}} \leftarrow \mathrm{triu}(\mathbf{1}_{B_r \times B_c}, d_R)$;
else if $d_L \geq -B_c$ **then**
    $M_{\mathrm{bool}} \leftarrow \mathrm{tril}(\mathbf{1}_{B_r \times B_c}, d_L)$;
else
    $M_{\mathrm{bool}} \leftarrow \mathbf{0}$;
end
$M \leftarrow \mathbf{0}$; $M[M_{\mathrm{bool}}] \leftarrow -\infty$;
return M;

A.2.1 Reference PyTorch Implementation

```
import torch

def generate_mask(r, c, Br, Bc, wl, wr, device="cpu"):
    delta = c * Bc - r * Br
    d_right = int(wr + delta) + 1
    d_left  = int(-wl + delta) - 1
    ones = torch.ones(Br, Bc, dtype=torch.bool, device=device)
    both = (d_right <= Br) and (d_left >= -Bc)
```

```
    if both:
        mask_bool = torch.triu(ones, diagonal=d_right) | \
                    torch.tril(ones, diagonal=d_left)
    elif d_right <= Br:
        mask_bool = torch.triu(ones, diagonal=d_right)
    elif d_left >= -Bc:
        mask_bool = torch.tril(ones, diagonal=d_left)
    else:
        mask_bool = torch.zeros_like(ones)
    mask = torch.zeros(Br, Bc, dtype=torch.float32, device=device)
    mask.masked_fill_(mask_bool, float("-inf"))
    return mask
```

A.3 Full SWFA Forward Pass

Algorithm A.3: SWFA-FORWARD: full expanded reference.

Input: Q, K, V; (w_l, w_r); (B_r, B_c); $\varepsilon > 0$.

Output: O.

$(\mathcal{I}, \mathcal{X}, \mathcal{O}) \leftarrow \textsc{ClassifyBlocks}(N, w_l, w_r, B_r, B_c)$;
for $r = 0$ **to** $T_r - 1$ **do**
  $Q_r \leftarrow Q[rB_r : (r+1)B_r]$;
  $O_r \leftarrow \mathbf{0}_{B_r \times d_v}$; $m_r \leftarrow -\infty$; $\ell_r \leftarrow 0$;
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $K_c \leftarrow K[cB_c : (c+1)B_c]$; $V_c \leftarrow V[cB_c : (c+1)B_c]$;
    $S \leftarrow (Q_r K_c^\top)/\sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $M \leftarrow \textsc{GenerateMask}(r, c, B_r, B_c, w_l, w_r)$;
      $S \leftarrow S + M$;
    end
    $m' \leftarrow \max(m_r, \max(S, \text{axis=1}))$;
    $\text{corr} \leftarrow \exp(m_r - m')$;
    $P \leftarrow \exp(S - m')$;
    $\ell_r \leftarrow \text{corr} \cdot \ell_r + \sum P$;
    $O_r \leftarrow \text{corr} \cdot O_r + PV_c$;
    $m_r \leftarrow m'$;
  end
  $O[rB_r : (r+1)B_r] \leftarrow O_r/(\ell_r + \varepsilon)$;
end
return O;

A.3.1 Reference PyTorch Implementation (CPU path)

```
def swfa_forward(Q, K, V, wl, wr, Br=16, Bc=16, eps=1e-6):
    """Single-head sliding-window flash attention (CPU reference).
    Q, K: [N, d];  V: [N, dv]
    Returns O: [N, dv]
```

```
    """
    N, d   = Q.shape
    _, dv  = V.shape
    O = torch.zeros(N, dv, dtype=Q.dtype, device=Q.device)
    I, X, Oset = classify_blocks(N, wl, wr, Br, Bc)
    skip = set(Oset)
    inter = set(X)
    sqrt_d = d ** 0.5
    Tr = (N + Br - 1) // Br
    Tc = (N + Bc - 1) // Bc
    for r in range(Tr):
        Qr = Q[r*Br : min((r+1)*Br, N)]
        Or = torch.zeros(Qr.shape[0], dv, dtype=Q.dtype)
        mr = torch.full((Qr.shape[0], 1), float("-inf"))
        lr = torch.zeros(Qr.shape[0], 1)
        for c in range(Tc):
            if (r, c) in skip: continue
            Kc = K[c*Bc : min((c+1)*Bc, N)]
            Vc = V[c*Bc : min((c+1)*Bc, N)]
            S = (Qr @ Kc.T) / sqrt_d
            if (r, c) in inter:
                M = generate_mask(r, c, Qr.shape[0], Kc.shape[0],
                                  wl, wr, device=Q.device)
                S = S + M
            m_new = torch.maximum(mr, S.max(dim=1, keepdim=True).values)
            corr  = torch.exp(mr - m_new)
            P     = torch.exp(S - m_new)
            lr    = corr * lr + P.sum(dim=1, keepdim=True)
            Or    = corr * Or + P @ Vc
            mr    = m_new
        O[r*Br : min((r+1)*Br, N)] = Or / (lr + eps)
    return O
```

A.4 Full SWFA Backward Pass

Algorithm A.4: SWFA-BACKWARD: full expanded reference.

Input: Q, K, V; saved (m_r, ℓ_r, O_r) for every r; $\mathrm{d}O$; $(\mathcal{I}, \mathcal{X}, \mathcal{O})$.

Output: $\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V$.

; // first sweep: precompute $\rho_r = \sum_c P_{rc} \odot (\mathrm{d}O_r V_c^\top)$

for $r = 0$ **to** $T_r - 1$ **do**
  $\rho_r \leftarrow \mathbf{0}$;
  for c ***with*** $(r, c) \notin \mathcal{O}$ **do**
    $S \leftarrow (Q_r K_c^\top)/\sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $S \leftarrow S + M_{rc}$;
    end
    $P \leftarrow \exp(S - m_r)/\ell_r$;
    $\rho_r \leftarrow \rho_r + \mathrm{sum}_{\mathrm{row}}(P \odot (\mathrm{d}O_r V_c^\top))$;
  end
end

; // second sweep: accumulate gradients

for $r = 0$ **to** $T_r - 1$ **do**
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue
    end
    $S \leftarrow (Q_r K_c^\top)/\sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      $S \leftarrow S + M_{rc}$;
    end
    $P \leftarrow \exp(S - m_r)/\ell_r$;
    $\mathrm{d}V_c \mathrel{+}= P^\top \mathrm{d}O_r$;
    $\mathrm{d}P \leftarrow \mathrm{d}O_r V_c^\top$;
    $\mathrm{d}S \leftarrow P \odot (\mathrm{d}P - \rho_r)$;
    $\mathrm{d}Q_r \mathrel{+}= \mathrm{d}S\, K_c/\sqrt{d}$;
    $\mathrm{d}K_c \mathrel{+}= \mathrm{d}S^\top Q_r/\sqrt{d}$;
  end
end

return $(\mathrm{d}Q, \mathrm{d}K, \mathrm{d}V)$;

A.5 Distributed-Training Reference Scripts

The DP/DDP/FSDP/ZeRO-3 skeletons used in Chapter 3 are collected below.

A.5.1 DDP Launcher

```
import os, torch, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

def main():
    local_rank = int(os.environ["LOCAL_RANK"])
    dist.init_process_group("nccl")
    torch.cuda.set_device(local_rank)
    model = build_model().cuda(local_rank)
    model = DDP(model, device_ids=[local_rank],
                bucket_cap_mb=25, find_unused_parameters=False)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
    for step, batch in enumerate(loader):
        out  = model(batch.x.cuda(local_rank))
        loss = loss_fn(out, batch.y.cuda(local_rank))
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()

if __name__ == "__main__":
    main()
```

A.5.2 FSDP Launcher

```
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP,
    MixedPrecision,
    ShardingStrategy)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
import functools

mp_policy = MixedPrecision(param_dtype=torch.float16,
                           reduce_dtype=torch.float32,
                           buffer_dtype=torch.float16)

wrap_policy = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={TransformerBlock})

model = FSDP(build_model().cuda(local_rank),
             auto_wrap_policy=wrap_policy,
             mixed_precision=mp_policy,
             sharding_strategy=ShardingStrategy.FULL_SHARD,
```

```
            device_id=local_rank)
```

A.5.3 DeepSpeed ZeRO-3 Configuration

```
{
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
    "contiguous_gradients": true,
    "stage3_prefetch_bucket_size": 2e8,
    "stage3_param_persistence_threshold": 1e6,
    "stage3_max_live_parameters": 1e9,
    "stage3_max_reuse_distance": 1e9,
    "reduce_bucket_size": 5e8
  },
  "fp16": { "enabled": true, "loss_scale_window": 1000,
            "initial_scale_power": 16 },
  "optimizer": {
    "type": "AdamW",
    "params": { "lr": 3e-4, "betas": [0.9, 0.999], "eps": 1e-8 }
  },
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 1,
  "gradient_clipping": 1.0,
  "wall_clock_breakdown": false
}
```

A.6 CUTLASS Kernel Snippets

A.6.1 GEMM + Bias + GELU Epilogue

```
using Gemm = cutlass::gemm::device::GemmUniversal<
    cutlass::half_t, cutlass::layout::RowMajor,
    cutlass::half_t, cutlass::layout::ColumnMajor,
    cutlass::half_t, cutlass::layout::RowMajor,
    float,                              // accumulator
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    cutlass::gemm::GemmShape<128,128,32>,
    cutlass::gemm::GemmShape< 64, 64,32>,
    cutlass::gemm::GemmShape< 16,  8,16>,
    cutlass::epilogue::thread::LinearCombinationGELU<
        cutlass::half_t, 8, float, float>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    3                                   // num pipeline stages
>;
```

A.6.2 Grouped GEMM Launch

```
using GroupGemm = cutlass::gemm::device::GemmGrouped<
    cutlass::half_t, cutlass::layout::RowMajor,
    cutlass::half_t, cutlass::layout::ColumnMajor,
    cutlass::half_t, cutlass::layout::RowMajor,
    float,
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    cutlass::gemm::GemmShape<128,128,32>,
    cutlass::gemm::GemmShape< 64, 64,32>,
    cutlass::gemm::GemmShape< 16,  8,16>,
    cutlass::epilogue::thread::LinearCombination<
        cutlass::half_t, 8, float, float>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    4                                 // num experts per launch
>;
```

A.7 Triton SWFA Kernel

The Triton implementation of SWFA used for the cross-validation in Section 7.5 is reproduced below in full.

```
import triton
import triton.language as tl

@triton.autotune(
    configs=[
        triton.Config({'BLOCK_R': 64,  'BLOCK_C': 64},
                      num_warps=4, num_stages=2),
        triton.Config({'BLOCK_R': 64,  'BLOCK_C': 128},
                      num_warps=4, num_stages=2),
        triton.Config({'BLOCK_R': 128, 'BLOCK_C': 64},
                      num_warps=8, num_stages=3),
    ],
    key=['N', 'd', 'wl', 'wr'])
@triton.jit
def swfa_fwd_kernel(Q, K, V, O, L, M,
                    block_idx, n_active,
                    stride_qn, stride_qd,
                    stride_kn, stride_kd,
                    stride_vn, stride_vd,
                    stride_on, stride_od,
                    N, d, wl, wr,
                    BLOCK_R: tl.constexpr,
                    BLOCK_C: tl.constexpr):
    pid = tl.program_id(0)
    if pid >= n_active:
        return
    rc = tl.load(block_idx + pid)
```

```
    r = rc // 65536
    c = rc %  65536

    # Load Q tile
    offs_r = r * BLOCK_R + tl.arange(0, BLOCK_R)
    offs_c = c * BLOCK_C + tl.arange(0, BLOCK_C)
    offs_d = tl.arange(0, d)
    Q_tile = tl.load(Q + offs_r[:, None]*stride_qn +
                         offs_d[None, :]*stride_qd,
                     mask=(offs_r[:, None] < N))

    K_tile = tl.load(K + offs_c[:, None]*stride_kn +
                         offs_d[None, :]*stride_kd,
                     mask=(offs_c[:, None] < N))

    # Scores
    scale = 1.0 / tl.sqrt(tl.full([1], d, dtype=tl.float32))
    S = tl.dot(Q_tile, tl.trans(K_tile)) * scale

    # Sliding-window mask (only for intersecting blocks)
    delta = c * BLOCK_C - r * BLOCK_R
    rel   = offs_c[None, :] - offs_r[:, None] - delta
    in_win = (rel <= wr) & (rel >= -wl)
    S = tl.where(in_win, S, float("-inf"))

    # Online softmax
    m_new = tl.max(S, axis=1)
    P     = tl.exp(S - m_new[:, None])
    l_new = tl.sum(P, axis=1)

    V_tile = tl.load(V + offs_c[:, None]*stride_vn +
                         offs_d[None, :]*stride_vd,
                     mask=(offs_c[:, None] < N))
    O_tile = tl.dot(P.to(V_tile.dtype), V_tile) / l_new[:, None]

    tl.store(O + offs_r[:, None]*stride_on +
                 offs_d[None, :]*stride_od, O_tile,
             mask=(offs_r[:, None] < N))
```

A.8 Numerical Validation Harness

The harness used to produce Table 5.1 follows.

```
import torch, math

def vanilla_attn(Q, K, V, wl, wr):
    N, d = Q.shape
    S = (Q @ K.T) / math.sqrt(d)
    i = torch.arange(N)[:, None]
    j = torch.arange(N)[None, :]
    mask = (j - i > wr) | (i - j > wl)
```

```
    S = S.masked_fill(mask, float("-inf"))
    P = torch.softmax(S, dim=-1)
    return P @ V

def validate(N=32, d=16, wl=4, wr=4, Br=8, Bc=8, seed=42, dtype=torch.float32):
    torch.manual_seed(seed)
    Q = torch.randn(N, d,  dtype=dtype)
    K = torch.randn(N, d,  dtype=dtype)
    V = torch.randn(N, d,  dtype=dtype)
    O_ref = vanilla_attn(Q, K, V, wl, wr)
    O_swf = swfa_forward(Q, K, V, wl, wr, Br, Bc)
    return (O_ref - O_swf).abs().max().item(), \
           (O_ref - O_swf).abs().mean().item()
```

Appendix B

Mathematical Proofs

This appendix collects the full proofs of the theoretical results stated in the main text. The proofs are self-contained: each reproduces the claim before giving the argument, and each is numbered so that it can be cited from any chapter.

B.1 Notation and Conventions

Throughout this appendix we use the following conventions.

- N denotes the sequence length; d the per-head embedding dimension; H the number of heads; B the batch size.
- (B_r, B_c) are the row/column block sizes of Flash Attention; $T_r = \lceil N/B_r \rceil$ and $T_c = \lceil N/B_c \rceil$.
- (w_l, w_r) are the left/right window half-widths; the full window is $w = w_l + w_r + 1$.
- $Q, K \in \mathbb{R}^{N \times d}$ and $V \in \mathbb{R}^{N \times d_v}$ are the query/key/value matrices.
- $A_{ij} \triangleq q_i \cdot k_j / \sqrt{d}$ is the raw attention score; the softmax-normalised weight is P_{ij}.

B.2 Correctness of Corner-Based Block Classification

Lemma B.1 (Restatement of Lemma 5.1)**.** *Let $B = [r_0, r_1] \times [c_0, c_1]$ be a block of the attention matrix. Define the window band $W = \{(i, j) \in [0, N-1]^2 : -w_l \le j - i \le w_r\}$. Then:*

(i) $B \cap W = \emptyset$ *iff* $c_1 < r_0 - w_l$ ***or*** $c_0 > r_1 + w_r$*;*

(ii) $B \subseteq W$ *iff* $c_1 \le r_0 + w_r$ ***and*** $c_0 \ge r_1 - w_l$.

Proof. Consider the scalar function $\phi(i, j) = j - i$ on B. Because ϕ is linear and B is a rectangle aligned with the coordinate axes, ϕ attains its minimum at the corner (r_1, c_0) (largest i, smallest j) and its maximum at (r_0, c_1) (smallest i, largest j).

(i) Outside case. $B \cap W = \emptyset$ iff ϕ never lies in $[-w_l, w_r]$ on B, which is equivalent to $\max_B \phi < -w_l$ *or* $\min_B \phi > w_r$. Substituting the extrema, $\max_B \phi = c_1 - r_0$ and $\min_B \phi = c_0 - r_1$, we get $c_1 - r_0 < -w_l$ or $c_0 - r_1 > w_r$, i.e. $c_1 < r_0 - w_l$ or $c_0 > r_1 + w_r$.

(ii) Inside case. $B \subseteq W$ iff $\phi(B) \subseteq [-w_l, w_r]$, which requires $\max_B \phi \le w_r$ and $\min_B \phi \ge -w_l$. Substituting, $c_1 - r_0 \le w_r$ and $c_0 - r_1 \ge -w_l$, i.e. $c_1 \le r_0 + w_r$ and $c_0 \ge r_1 - w_l$.

The clipping of W to $[0, N-1]^2$ does not change either inequality, because B itself is a subset of $[0, N-1]^2$ by construction. □

Corollary B.1. *Algorithm 5.1 runs in $\mathcal{O}(T_r T_c)$ arithmetic operations and correctly assigns each block to exactly one of the sets $\mathcal{I}, \mathcal{X}, \mathcal{O}$.*

Proof. For each of the $T_r T_c$ blocks, Algorithm 5.1 performs a constant number of comparisons derived from the two predicates of Lemma B.1. The three sets are disjoint by construction: the `outside` predicate takes precedence; the `inside` predicate is only tested when `outside` is false; and the remaining blocks are placed in $\mathcal{X}$. By Lemma B.1, a block is outside iff `outside` is true, and inside iff `inside` is true; the three sets are therefore a partition of the grid. □

B.3 Block-Count Lemma

Lemma B.2 (Skipped-block count)**.** *For a symmetric window $w_l = w_r = W$ and equal block sizes $B_r = B_c = B$, with $B \mid N$, the number of outside blocks is*

$$|\mathcal{O}| \;=\; 2 \sum_{k=1}^{T_r - 1} \max(0, T_r - k - \lceil (W + (k-1)B)/B \rceil) \;=\; (T_r)^2 - \Theta(T_r W/B) \quad \text{(B.1)}$$

and therefore the skip ratio converges to $1 - 2W/N$ as $N \to \infty$ with fixed W.

Proof. With $B \mid N$, the grid has $T = N/B$ blocks per axis. By Lemma B.1(i), block (r, c) is outside iff $cB + B - 1 < rB - W$ or $cB > rB + B - 1 + W$, i.e.

$$c \le r - \lceil (W + B - 1)/B \rceil \quad \text{or} \quad c \ge r + \lceil (W + B - 1)/B \rceil \,. \quad \text{(B.2)}$$

Let $k^\star = \lceil (W+B-1)/B \rceil$. The number of outside blocks in row r is therefore $\max(0, r - k^\star) + \max(0, T-1-r-k^\star)$, and summing over r yields $|\mathcal{O}| = 2\sum_{r=0}^{T-1} \max(0, r-k^\star)$ after collecting symmetric terms, which simplifies to $T^2 - (2k^\star - 1)T + k^\star(k^\star - 1)$. Dividing by T^2 and letting $N \to \infty$ with W fixed, $k^\star/T \to W/N$, and the skip ratio converges to $1 - 2W/N$, as claimed. □

B.4 Convexity of the SWFA Cost

Theorem B.1 (Restatement of Theorem 5.1)**.** *The cost function*

$$\mathcal{L}(w) = \alpha\kappa N w d + \beta M_0 + \frac{2\gamma N A_0}{1 - e^{-\lambda}}\, e^{-\lambda w/2} \tag{B.3}$$

is strictly convex for $w > 0$*.*

Proof. Differentiating twice,

$$\mathcal{L}'(w) = \alpha\kappa N d - \frac{\gamma N A_0 \lambda}{1 - e^{-\lambda}} e^{-\lambda w/2}, \tag{B.4}$$

$$\mathcal{L}''(w) = \frac{\gamma N A_0 \lambda^2}{2(1 - e^{-\lambda})} e^{-\lambda w/2}. \tag{B.5}$$

All prefactors are positive: $\gamma > 0$ by assumption, $N, A_0, \lambda > 0$ by model specification, and $1 - e^{-\lambda} > 0$ because $\lambda > 0$. The exponential $e^{-\lambda w/2} > 0$ for all $w \in \mathbb{R}$. Hence $\mathcal{L}''(w) > 0$ for all $w > 0$, and $\mathcal{L}$ is strictly convex. □

Remark B.1. Because $\mathcal{L}''$ is a positive exponential times a constant, the convexity is *strong*: for any $w_0 > 0$ there exists $\mu > 0$ such that $\mathcal{L}''(w) \geq \mu$ on any bounded interval containing w_0. This gives a linear convergence rate for gradient descent on $\mathcal{L}$ and justifies a Newton step for the $w^\star$ derivation below.

B.5 Closed-Form Optimal Window

Theorem B.2 (Restatement of Theorem 5.2)**.** *The minimiser of* $\mathcal{L}$ *is*

$$w^\star = \frac{2}{\lambda} \ln\!\left(\frac{\gamma A_0 \lambda}{\alpha\kappa d (1 - e^{-\lambda})} \right), \tag{B.6}$$

provided the argument of the logarithm exceeds one (otherwise the unconstrained minimiser is non-positive and the projected minimiser equals 1*).*

Proof. By Theorem B.1, $\mathcal{L}$ is strictly convex on $w > 0$, so its minimiser is the unique stationary point. $\mathcal{L}'(w) = 0$ iff

$$\alpha\kappa N d = \frac{\gamma N A_0 \lambda}{1 - e^{-\lambda}} e^{-\lambda w/2}, \tag{B.7}$$

i.e.

$$e^{-\lambda w/2} = \frac{\alpha\kappa d(1 - e^{-\lambda})}{\gamma A_0 \lambda}. \tag{B.8}$$

Taking $\ln$ of both sides and multiplying by $-2/\lambda$ gives (B.6). The argument of the outer logarithm is positive, so $w^\star \in \mathbb{R}$; it is positive exactly when the argument exceeds one, which is the regime in which local attention actually helps (otherwise dense attention is cheaper than the loss in accuracy and $w = N - 1$ is optimal).

Projection onto $[1, N - 1]$ preserves uniqueness: on the left, the convexity implies $\mathcal{L}(1) \leq \mathcal{L}(0)$, and on the right $\mathcal{L}(N - 1)$ is dominated by the computation term because the exponential has decayed. Hence the projected minimiser is obtained by clipping $w^\star$ to $[1, N - 1]$. □

Corollary B.2 (Sensitivities)**.** *Let* $w^\star$ *be as in Theorem B.2. Then:*

(i) $\partial w^\star / \partial \alpha < 0$*;*

(ii) $\partial w^\star / \partial \gamma > 0$*;*

(iii) $\partial w^\star / \partial d < 0$*;*

(iv) $w^\star \to \infty$ *as* $\lambda \to 0^+$.

Proof. Taking $\partial/\partial\alpha$ of (B.6) gives $\partial w^\star/\partial\alpha = -2/(\lambda\alpha) < 0$. Similarly, $\partial w^\star/\partial\gamma = 2/(\lambda\gamma) > 0$ and $\partial w^\star/\partial d = -2/(\lambda d) < 0$. For (iv), expand the logarithm for small λ: $1 - e^{-\lambda} \approx \lambda - \lambda^2/2$, so the argument of the outer log tends to $\gamma A_0/(\alpha\kappa d)$, which is $O(1)$. The outer factor $2/\lambda$ diverges, so $w^\star \to \infty$. □

B.6 Flash Attention Softmax Correctness

The online softmax update used by both Flash Attention and SWFA is algebraically equivalent to the full-batch softmax. The proof is a straightforward induction, reproduced for completeness.

Lemma B.3 (Online softmax equivalence)**.** *Given scalars $s_1, \dots, s_N$ and values $v_1, \dots, v_N$, let $m = \max_i s_i$, $\ell = \sum_i e^{s_i - m}$, and $o = \sum_i e^{s_i - m} v_i / \ell$. Then for any partition of the indices into T blocks, the iterative update*

$$m^{(t)} = \max(m^{(t-1)}, \max_{i \in B_t} s_i), \tag{B.9}$$

$$\ell^{(t)} = e^{m^{(t-1)} - m^{(t)}} \ell^{(t-1)} + \sum_{i \in B_t} e^{s_i - m^{(t)}}, \tag{B.10}$$

$$o^{(t)} = \frac{e^{m^{(t-1)} - m^{(t)}} \ell^{(t-1)} o^{(t-1)} + \sum_{i \in B_t} e^{s_i - m^{(t)}} v_i}{\ell^{(t)}}, \tag{B.11}$$

with $(m^{(0)}, \ell^{(0)}, o^{(0)}) = (-\infty, 0, 0)$, yields $(m^{(T)}, \ell^{(T)}, o^{(T)}) = (m, \ell, o)$.

Proof. By induction on t. Denote by $I_t = \bigcup_{s \le t} B_s$ the set of indices processed up to step t, and define $m_t = \max_{i \in I_t} s_i$, $\ell_t = \sum_{i \in I_t} e^{s_i - m_t}$, $o_t = \sum_{i \in I_t} e^{s_i - m_t} v_i / \ell_t$.

Base case. $t = 0$: $I_0 = \emptyset$, all three quantities coincide with the initialisation.

Inductive step. Assume the claim holds for $t-1$. At step t, $m^{(t)} = \max(m^{(t-1)}, \max_{i \in B_t} s_i) = m_t$ by definition. Multiplying the previous sum by $e^{m^{(t-1)} - m^{(t)}}$ rescales the exponents to the new reference $m^{(t)}$, and adding the contribution from B_t produces exactly $\ell_t = \sum_{i \in I_t} e^{s_i - m^{(t)}}$, matching the claim. The same rescaling applied to the numerator of o gives o_t. □

Corollary B.3 (SWFA equivalence)**.** *SWFA produces the same output as vanilla sliding-window attention up to floating-point round-off.*

Proof. SWFA skips exactly the blocks that are outside the window (by Lemma B.1(i)), and on the remaining blocks it applies either no mask (inside) or the additive $\{0, -\infty\}$ mask of Algorithm 5.2 (intersecting). Because masked entries contribute zero to both the numerator and the denominator of the softmax, the online update of Lemma B.3 acting on masked scores is equivalent to the online update acting on the full set of in-window scores. Hence SWFA's output equals the vanilla sliding-window attention's output up to the rearrangement of the summation order, and floating-point associativity accounts for any observed differences. □

B.7 Communication-Cost Bounds for Distributed Strategies

Lemma B.4 (DDP ring-allreduce bandwidth)**.** *On a ring topology with P workers and link bandwidth B, a single all-reduce of a tensor of size Ms bytes takes*

$$T_{ring}(M, P) = 2(P-1)\alpha + \frac{2(P-1)Ms}{PB}, \tag{B.12}$$

where α is the per-hop latency and the factor $2(P-1)/P$ reflects the two phases (reduce-scatter + all-gather).

Proof. A ring all-reduce proceeds in two phases of $P-1$ steps each. Each step sends a fragment of size Ms/P to the next worker. The latency term is therefore $2(P-1)\alpha$, and the bandwidth term is $2(P-1)(Ms/P)/B = 2(P-1)Ms/(PB)$. Summing gives (B.12). □

Corollary B.4. *As $P \to \infty$ with M fixed, $T_{ring}(M, P) \to 2Ms/B + 2(P-1)\alpha$, so the per-worker bandwidth term is asymptotically independent of P and the scalability is limited only by the latency constant.*

Lemma B.5 (ZeRO-3 communication volume)**.** *ZeRO-3 incurs, per training step, a communication volume of*

$$V_{ZeRO\text{-}3} = \frac{2(P-1)}{P} \cdot Ms + \frac{2(P-1)}{P} \cdot Ms + \frac{2(P-1)}{P} \cdot Ms_{grad} \tag{B.13}$$

(two all-gathers for forward/backward parameters plus one reduce-scatter for gradients).

Proof. Each parameter is sharded across P workers. Before the forward pass, each worker must receive the other workers' shards via an all-gather, contributing $(P-1)/P \cdot Ms$ per worker-input. By symmetry the reverse all-gather before the backward pass adds a second $(P-1)/P \cdot Ms$; the reduce-scatter of the gradients is the symmetric operation and contributes $(P-1)/P \cdot Ms_{\text{grad}}$. Summing gives the stated total. □

Remark B.2. Comparing Lemma B.5 with Lemma B.4, ZeRO-3 sends roughly $1.5\times$ the data of DDP per step. The overhead is amortised by the $8\times$ reduction in memory, which allows a correspondingly larger batch size and therefore fewer steps per epoch.

B.8 Exponential Decay Assumption

The attention-decay assumption $A_{ij} = A_0 e^{-\lambda|i-j|}$ is a modelling choice. We justify it from two directions.

Proposition B.1 (Empirical decay)**.** *For a transformer with softmax attention and inputs drawn from a stationary Markov process with mixing time τ, the expected attention weight between tokens at distance $\Delta = |i - j|$ satisfies $\mathbb{E}[P_{ij}] \leq C \cdot e^{-\Delta/\tau}$ for some constant $C > 0$.*

Proof sketch. By the data processing inequality for the Markov chain, the mutual information $I(x_i; x_j)$ decays exponentially in $|i - j|$ with rate $1/\tau$. Because softmax attention can be viewed as a non-parametric estimator of $I(x_i; x_j)$ up to a constant, its expectation also decays at rate $1/\tau$. Setting $\lambda = 1/\tau$ yields the claim; a rigorous statement requires Dobrushin-style contraction inequalities, which we omit. □

Remark B.3. The closed-form optimal-window formula (B.6) is robust to misspecification of λ: the logarithm dampens parameter misestimation, so halving or doubling λ changes $w^\star$ by at most $\ln 2 \cdot 2/\lambda$, which is typically $< 20\%$ of the value itself.

B.9 Reproducibility of the Numerical Accuracy Bound

Proposition B.2 (FP32 error bound for SWFA)**.** *Let u denote the FP32 unit round-off ($u \approx 6 \times 10^{-8}$). The SWFA output $\hat{O}$ satisfies*

$$\|O - \hat{O}\|_\infty \leq N(3d + 2)u \cdot \|Q\|_\infty \|K\|_\infty \|V\|_\infty + \mathcal{O}(u^2). \tag{B.14}$$

Proof sketch. Each entry of $S_{rc} = Q_r K_c^\top / \sqrt{d}$ is the inner product of two d-vectors and incurs relative error at most du. The online softmax adds at most two multiplications and one subtraction, contributing $3u$ per block pass; with T_c column blocks and sub-additive rounding, the total is $3d + 2T_c u \leq 3d + 2N/B_c u$. Multiplying by V introduces another d-vector inner product, bounded by $\|V\|_\infty$. Combining the bounds gives the claim up to $\mathcal{O}(u^2)$ terms. □

Proposition B.2 predicts a worst-case FP32 error of $\sim 4 \times 10^{-5}$ at $N = 128$, $d = 16$, which comfortably bounds the measured $\sim 8 \times 10^{-7}$ in Table 5.1. The gap is explained by the typical random-input pessimism of the analysis.